

Laporan Tugas Besar

Arion Compiler

IF2224 Teori Bahasa Formal dan Otomata



Oleh :
khusus 1352____

Daniel Anindito Nugroho 13524002

Ishak Palentino Napitupulu 13524022

Al Farabi 13524086

Ishaq Irfan Farizal 13524094

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

I. Teori Singkat

A. Analisis Semantik

Analisis semantik merupakan tahap ketiga dalam proses kompilasi atau interpretasi, setelah analisis leksikal dan analisis sintaksis. Setelah parser memastikan bahwa struktur program sesuai dengan tata bahasa (sintaksis) yang telah ditetapkan, analisis semantik bertugas untuk memverifikasi kepatuhan program terhadap aturan-aturan makna (semantik) dari bahasa pemrograman. Aturan-aturan tersebut mencakup, antara lain:

- Setiap identifier (variabel, konstanta, prosedur, fungsi) harus telah dideklarasikan sebelum digunakan
- Tidak boleh ada deklarasi ulang identifier dalam cakupan (scope) yang sama
- Kesesuaian tipe data antara operand dalam suatu ekspresi maupun antara nilai dan variabel target dalam penugasan (assignment)
- Variabel harus telah diinisialisasi sebelum dibaca
- Validitas pemanggilan prosedur dan fungsi (jumlah dan tipe parameter)
- Validitas akses terhadap elemen array dan field record
- Serta kepatuhan alur kontrol (misalnya ekspresi kondisi pada if atau while harus bertipe Boolean)

Analisis semantik memastikan bahwa program tidak hanya memiliki struktur yang benar secara sintaksis, tetapi juga bermakna secara logis menurut spesifikasi bahasa.

B. Abstract Syntax Tree (AST)

Abstract Syntax Tree (AST) merupakan representasi hierarkis dari program yang telah disederhanakan dengan cara menghilangkan elemen-elemen sintaksis yang tidak diperlukan untuk keperluan analisis makna. Elemen-elemen yang dihilangkan antara lain tanda baca (seperti titik koma), tanda kurung, serta kata kunci pembatas blok seperti begin dan end. Struktur AST hanya mempertahankan operator dan operand sebagai inti logika program, sehingga bentuknya lebih ringkas dibandingkan pohon parse (parse tree). Karena bentuknya yang ringkas, AST menjadi format data yang efisien untuk proses penelusuran alur logika program, pengecekan tipe data, serta persiapan dalam pembangkitan kode eksekusi pada tahap-tahap selanjutnya.

C. L-Attributed Grammar dan Syntax-Directed Translation

Spesifikasi Milestone 3 menyebutkan penggunaan L-Attributed Grammar dan Syntax-Directed Translation dengan fungsi-fungsi visit. Dalam implementasi tugas ini, pendekatan yang diambil adalah sebagai berikut. Parser menghasilkan pohon parse (parse tree) sesuai dengan tata bahasa Arion. Selanjutnya, modul AstBuilder mengonversi pohon parse tersebut menjadi AST dengan menerapkan gagasan Syntax-Directed Translation: setiap aturan produksi secara implisit dikaitkan dengan pembentukan simpul AST yang sesuai. Setelah AST terbentuk, proses analisis semantik dilakukan secara terpisah oleh

SemanticAnalyzer, yang menelusuri AST menggunakan fungsi-fungsi visitor. Setiap fungsi `visit_<node_type>` bertanggung jawab untuk:

- Memanggil visit pada node-node anak (jika ada)
- Mengakses dan memodifikasi tabel simbol (penambahan entri, pencarian, pengelolaan scope)
- Melakukan pengecekan tipe data dan aturan semantik lainnya
- Menambahkan anotasi (tipe data, indeks tabel simbol, tingkat leksikal, dll.) ke node AST

Pendekatan dua tahap ini (konversi pohon parse ke AST, kemudian dekorasi AST dengan visitor) merupakan implementasi dari prinsip Syntax-Directed Translation dan L-Attributed Grammar yang diminta dalam spesifikasi.

D. Tabel Simbol (Identifier Table)

Tabel simbol, yang dalam spesifikasi ini disebut `tab`, berfungsi untuk menyimpan informasi setiap identifier yang ditemukan dalam kode sumber. Setiap variabel, konstanta, prosedur, fungsi, dan tipe data dicatat dalam tabel ini beserta atribut-atributnya, yang meliputi:

- `identifiers`: nama identifier
- `obj`: kategori objek (konstanta, variabel, tipe, prosedur, fungsi, program, parameter, atau field)
- `type`: tipe data (merupakan indeks ke tipe yang bersangkutan)
- `ref`: indeks referensi untuk tipe komposit (menunjuk ke `atab` atau `btab`)
- `nrm`: tanda variabel normal (1) atau parameter by-reference (0)
- `lev`: tingkat leksikal tempat identifier dideklarasikan
- `adr`: alamat memori (offset) atau nilai konstanta
- `link`: tautan ke identifier lain dalam blok yang sama (untuk membentuk rantai)
- `initialized`: tanda apakah variabel telah diberi nilai awal (digunakan untuk pemeriksaan inisialisasi).

Pencarian identifier dalam `tab` dilakukan dengan menelusuri rantai link dari blok terdalam hingga blok global. Perlu dicatat bahwa sifat case-insensitive (tidak membedakan huruf besar-kecil) hanya diberlakukan sepenuhnya untuk identifier bawaan (`predefined/reserved`). Untuk identifier yang didefinisikan oleh pengguna, huruf besar dan kecil diperlakukan sebagai berbeda (misalnya `x` dan `X` adalah dua identifier yang berbeda).

E. Tabel Blok (Block Table)

Struktur `btab` (block table) digunakan untuk mengelola informasi mengenai batasan-batasan lingkup (scope) dalam program, khususnya pada blok-blok yang bersifat bersarang seperti prosedur, fungsi, atau definisi tipe record. Setiap entri dalam `btab` mencatat:

- last: indeks (dari tab) ke identifier terakhir yang dideklarasikan dalam blok tersebut
- lpar: indeks ke parameter terakhir (jika blok merupakan prosedur atau fungsi; untuk blok record bernilai 0)
- psze: total ukuran memori untuk parameter-parameter dalam blok
- vsze: total ukuran memori untuk variabel lokal dalam blok

Tabel ini diperlukan oleh compiler untuk menangani akses variabel pada prosedur/fungsi yang bersarang (nested procedures) serta untuk mengalokasikan ruang memori yang tepat bagi setiap blok.

F. Tabel Array (Array Table)

Tipe data array memiliki karakteristik yang khas, yaitu memiliki indeks dengan batas bawah dan batas atas, serta elemen dengan tipe tertentu. Informasi-informasi ini disimpan dalam atab (array table). Setiap entri dalam atab mencatat:

- xtyp: tipe indeks array (merupakan indeks ke tipe dalam tab)
- etyp: tipe elemen array
- eref: indeks ke atab atau btap jika elemen berupa tipe komposit (array bersarang atau record)
- low: batas bawah indeks
- high: batas atas indeks
- elsz: ukuran satu elemen (dalam satuan memori)
- size: total ukuran array.

Dalam implementasi saat ini, informasi atab terbangun dengan baik untuk array yang dideklarasikan menggunakan rentang (range), misalnya array[1..10] of Integer. Untuk array dengan tipe indeks berupa tipe ordinal non-rentang (misalnya array[Char] of Real), informasi batas bawah dan atas masih bersifat terbatas.

G. Kompatibilitas Tipe Data

Aturan kompatibilitas tipe data mendefinisikan batasan dan interaksi yang diperbolehkan antara berbagai tipe data dalam operasi aritmetika, operasi logika, maupun penugasan (assignment). Berdasarkan spesifikasi program, aturan yang diterapkan mencakup tiga aspek utama.

Pertama, type compatibility (kompatibilitas dua tipe, dua tipe T1 dan T2 dikatakan kompatibel jika salah satu kondisi berikut terpenuhi:

- T1 dan T2 adalah tipe yang sama
- salah satu merupakan subrange dari yang lain
- kedua tipe merupakan subrange dari tipe dasar yang sama
- kedua tipe adalah String dengan panjang yang sama

Kedua, assignment compatibility (kompatibilitas penugasan), suatu nilai bertipe T2 dapat diberikan ke variabel bertipe T1 jika salah satu kondisi berikut terpenuhi:

- T1 adalah Real dan T2 adalah Integer (promosi tipe)
- T1 dan T2 kompatibel, keduanya merupakan tipe ordinal (Integer, Boolean, Char), dan nilai T2 berada dalam rentang nilai T1
- T1 dan T2 kompatibel dan keduanya bertipe String

Ketiga, operator result type (tipe hasil operasi): tipe hasil suatu operasi ditentukan oleh tipe operandnya, misalnya:

- +, -, * pada dua Integer menghasilkan Integer
- +, -, * pada campuran Integer dan Real menghasilkan Real
- / (pembagian riil) menghasilkan Real
- div, mod menghasilkan Integer
- and, or, not menghasilkan Boolean
- operator relasional (==, <>, <, >, <=, >=) menghasilkan Boolean.

Dalam implementasi saat ini, aturan-aturan tersebut telah diterapkan secara konseptual, namun dengan beberapa keterbatasan. Pengecekan kompatibilitas panjang string belum ditegakkan sepenuhnya, dan pengecekan subrange (apakah suatu nilai berada dalam rentang) baru diterapkan secara terbatas untuk nilai-nilai konstanta, belum untuk ekspresi yang lebih kompleks.

H. Lingkup Leksikal

Lingkup leksikal mengatur masa hidup dan visibilitas suatu identifier berdasarkan posisi deklarasinya di dalam struktur program. Bahasa Arion menerapkan aturan static scoping, suatu identifier dikenali pada blok tempat ia dideklarasikan serta pada seluruh blok yang berada di dalamnya (blok anak), tetapi tidak dikenali di luar blok tersebut.

Compiler melacak lingkup leksikal dengan menggunakan tingkat leksikal (lev), di mana tingkat 0 menunjukkan blok global (program utama), tingkat 1 menunjukkan blok prosedur atau fungsi yang dideklarasikan di tingkat global, dan seterusnya. Implementasi saat ini mendukung nested scopes melalui struktur tumpukan (scope stack) yang dikelola selama penelusuran AST. Namun, sifat case-insensitive untuk identifier pengguna tidak diberlakukan, sehingga x dan X dianggap berbeda. Mekanisme ini memastikan bahwa variabel lokal hanya dapat diakses dalam blok tempat ia dideklarasikan, sehingga konflik penggunaan nama variabel yang sama pada prosedur yang berbeda dapat dihindari.

I. Predefined Identifiers

Agar pengguna dapat menulis kode secara langsung tanpa harus mendeklarasikan tipe-tipe dasar dan konstanta fundamental, compiler menyediakan sejumlah identifier bawaan (predefined identifiers). Identifier-identifier ini telah dikenali oleh compiler

sebelum proses analisis semantik dimulai dan dicatat dalam tabel simbol pada indeks-indeks awal.

Dalam implementasi tugas ini, reserved words (kata kunci seperti program, var, begin, end, dll.) ditempatkan pada indeks 0 hingga 31. Selanjutnya, pada saat inisialisasi tabel simbol, ditambahkan identifier bawaan berupa:

- tipe data: Integer, Real, Char, Boolean, String;
- konstanta logika: True, False;
- prosedur bawaan (opsional): readln, writeln.

Identifier-identifier ini dilindungi agar tidak dapat dideklarasikan ulang oleh pengguna dalam lingkup yang sama, karena akan menimbulkan ambiguitas dan melanggar spesifikasi bahasa. Identifier yang didefinisikan oleh pengguna (programmer) dimulai setelah indeks-indeks yang ditempati oleh reserved words dan identifier bawaan tersebut, karena jumlah identifier bawaan dapat bervariasi tergantung pada kebutuhan implementasi.

II. Perancangan & Implementasi

A. Arsitektur Umum Semantic Analysis

Pada Milestone 3, arsitektur program tidak berhenti pada parser, tetapi dilanjutkan ke semantic analysis. Alur frontend compiler adalah sebagai berikut: source code dibaca oleh lexer, token diteruskan ke parser, parser membentuk parse tree, AstBuilder mengubah parse tree menjadi AST ringkas, SemanticAnalyzer mendekorasi dan memvalidasi AST, lalu printer mencetak Decorated AST, Symbol Table, dan semantic errors.

Implementasi utama semantic analysis berada pada folder src/semantic. Modul ast.hpp mendefinisikan struktur node AST dan anotasinya. Modul ast_builder.cpp membangun AST dari parse tree. Modul semantic_analyzer.cpp, semantic_analyzer_declarations.cpp, semantic_analyzer_statements.cpp, dan semantic_analyzer_expressions.cpp menjalankan visitor semantik. Modul symbol_table.* mengelola tab, btab, atab, scope stack, dan predefined identifiers. Modul types.* menyimpan registry tipe dan aturan kompatibilitas. Modul printer.* mencetak laporan semantik.

B. Alur Program dan Integrasi Semantic Analyzer

Struktur program utama terdapat pada src/main.cpp. Program menerima satu argumen berupa path file input. File tersebut diproses oleh Lexer menjadi token. Jika ditemukan TOKEN_ERROR, program menghentikan parsing dan menampilkan lexical error.

Setelah token diperoleh, TOKEN_COMMENT dan TOKEN_NEWLINE difilter agar komentar dan newline tidak masuk ke parser. Token hasil filter diberikan ke Parser untuk menghasilkan parse tree. Parse tree dicetak pada bagian `=== Parse Tree ===`. Jika parser error ditemukan, program berhenti sebelum semantic analysis.

Jika parsing berhasil, parse tree diberikan kepada `semantic::AstBuilder`. Hasil AST kemudian diberikan kepada `semantic::SemanticAnalyzer`. Analyzer menghasilkan `SemanticResult` yang berisi Decorated AST, TypeRegistry, SymbolTable, dan diagnostic. Terakhir, `print_semantic_report()` mencetak hasil ke terminal dan menulis file output ke `test/milestone-3/<nama_input>_SEMANTIC.txt`.

C. Semantic Action dan Syntax-Directed Translation

Pada laporan Milestone 3, bagian ini digunakan untuk menjelaskan semantic action atau Syntax-Directed Translation yang diterapkan terhadap konstruksi grammar utama. Tabel berikut merangkum aksi semantik yang digunakan dalam implementasi.

Konstruksi grammar	Semantic action	AST node yang dibentuk	Atribut/anotasi yang dihasilkan
program	Ambil nama program dari header, bangun declaration part dan compound statement. Analyzer memasukkan nama program ke tab.	Program	tab_index, block_index, lexical_level
const declaration	Ambil identifier dan constant, evaluasi literal atau unary constant, lalu masukkan sebagai constant.	ConstDecl, Literal, UnaryOp	type_id, tab_index, initialized=true, value di tab
type declaration	Ambil identifier type dan node tipe. Analyzer menyelesaikan TypeRef, RangeType, EnumType, ArrayType, atau RecordType.	TypeDecl dan node tipe terkait	type_id, tab_index, ref ke atab atau btab bila komposit
var declaration	Pecah identifier list menjadi satu VarDecl per identifier. Analyzer memasukkan variable ke scope aktif.	VarDecl	type_id, tab_index, lexical_level, initialized=false
assignment	Bentuk target dan expression, evaluasi keduanya, cek target assignable dan assignment-compatible.	AssignStatement	type_id target, tab_index target, status inisialisasi target diperbarui
if	Bentuk kondisi dan statement cabang. Analyzer memastikan kondisi bertipe Boolean.	IfStatement	type_id=Boolean pada kondisi/statement jika valid
while	Bentuk kondisi dan body compound. Analyzer memastikan kondisi Boolean.	WhileStatement	type_id=Boolean; body diberi block_index dan lexical_level
repeat	Statement list disatukan menjadi CompoundStatement, lalu kondisi until dievaluasi.	RepeatStatement, CompoundStatement	type_id=Boolean pada kondisi jika valid
for	Simpan nama iterator, arah to/downto, bound awal, bound akhir, dan body compound.	ForStatement	tab_index iterator; iterator ditandai initialized bila

			valid
case	Bentuk selector dan kumpulan branch. Analyzer mengecek kompatibilitas label dengan selector.	CaseStatement, CaseBranch	type_id selector
procedure/function call	Ambil nama call dan argument expression. Analyzer mengecek callable, jumlah argument, dan tipe argument.	Call	tab_index callee, type_id return untuk function
expression	Susun operator sesuai precedence: relational, additive, multiplicative, unary. Analyzer menentukan tipe hasil.	UnaryOp, BinaryOp, Literal, Variable, Call	type_id, tab_index bila expression mengacu symbol
variable access	Ambil identifier dasar dan component-variable. Analyzer melakukan lookup dan cek inisialisasi.	Variable	type_id, tab_index, lexical_level, initialized
array indexing	Simpan daftar ekspresi indeks sebagai komponen variable. Analyzer memastikan target array dan indeks compatible.	IndexComponent	Tipe variable berubah menjadi tipe elemen array
record field access	Simpan nama field setelah operator titik. Analyzer memastikan target record dan field terdaftar.	FieldComponent	tab_index field, tipe variable berubah menjadi tipe field

D. Implementasi Semantic Analysis

1. Konversi Parse Tree menjadi AST

Parse tree tetap menjadi keluaran parser dan masukan bagi tahap semantic analysis. Pada Milestone 3, parse tree tidak langsung dianalisis secara semantik, melainkan diterjemahkan terlebih dahulu menjadi AST oleh AstBuilder. Tujuannya adalah mengurangi detail grammar yang tidak diperlukan untuk pengecekan makna.

Contoh informasi yang dihilangkan dari parse tree adalah keyword begin, end, tanda titik koma, tanda kurung sintaksis, dan wrapper produksi yang tidak berpengaruh pada makna program. Informasi yang dipertahankan adalah nama identifier, literal, operator, deklarasi, statement, dan struktur akses data.

```
ast_builder.cpp
```

```

AstNodePtr build_statement(const std::shared_ptr<ParseTreeNode>& node) {
    if (!node || node->children.empty()) return make_ast(AstKind::EmptyStatement,
        location_from_node(node));
    const auto& child = node->children.front();

    if (is_node(child, "<assignment-statement>")) {
        auto ast = make_ast(AstKind::AssignStatement, location_from_node(child));
        ast->add_child(build_variable(first_child(child, "<variable>")));
        ast->add_child(build_expression(first_child(child, "<expression>")));
        return ast;
    }
    if (is_node(child, "<if-statement>")) {
        auto ast = make_ast(AstKind::IfStatement, location_from_node(child));
        ast->add_child(build_expression(first_child(child, "<expression>")));
        for (const auto& item : child->children) {
            if (is_node(item, "<statement>")) ast->add_child(build_statement(item));
        }
        return ast;
    }
    if (is_node(child, "<while-statement>")) {
        auto ast = make_ast(AstKind::WhileStatement, location_from_node(child));
        ast->add_child(build_expression(first_child(child, "<expression>")));
        ast->add_child(build_compound_statement(first_child(child, "<compound-statement>")));
        return ast;
    }
    if (is_node(child, "<repeat-statement>")) {
        auto ast = make_ast(AstKind::RepeatStatement, location_from_node(child));
        auto body = make_ast(AstKind::CompoundStatement, location_from_node(child));
        for (auto& statement : build_statement_list(first_child(child, "<statement-list>"))) {
            body->add_child(statement);
        }
        ast->add_child(body);
        for (const auto& item : child->children) {
            if (is_node(item, "<expression>")) ast->add_child(build_expression(item));
        }
        return ast;
    }
    if (is_node(child, "<for-statement>")) {
        auto ast = make_ast(AstKind::ForStatement, location_from_node(child));
        for (const auto& item : child->children) {
            if (terminal_is(item, "ident") && ast->name.empty()) ast->name = terminal_value(item);
            else if (terminal_is(item, "tosy") || terminal_is(item, "downtosy")) ast->op =
                terminal_token(item);
            else if (is_node(item, "<expression>")) ast->add_child(build_expression(item));
            else if (is_node(item, "<compound-statement>")) ast->
                add_child(build_compound_statement(item));
        }
        return ast;
    }
    if (is_node(child, "<case-statement>")) {
        auto ast = make_ast(AstKind::CaseStatement, location_from_node(child));
        ast->add_child(build_expression(first_child(child, "<expression>")));
        append_case_branches(first_child(child, "<case-block>"), ast);
        return ast;
    }
    if (is_node(child, "<procedure/function-call>")) return build_call(child);
    if (is_node(child, "<compound-statement>")) return build_compound_statement(child);
    if (is_node(child, "<variables>")) return build_variable(child);
    return make_ast(AstKind::EmptyStatement, location_from_node(node));
}

```

2. Struktur AST dan Decorated AST

Parser tetap bertanggung jawab menghasilkan parse tree yang valid. Namun untuk Milestone 3, fokus pembahasan berpindah ke kelas dan struktur pada modul semantic. Struktur AST didefinisikan di src/semantic/ast.hpp melalui enum AstKind, enum LiteralKind, struct AstAnnotation, dan struct AstNode.

AstKind mencakup node penting seperti Program, DeclarationPart, ConstDecl, TypeDecl, VarDecl, ProcedureDecl, FunctionDecl, ParameterGroup,

CompoundStatement, EmptyStatement, AssignStatement, IfStatement, CaseStatement, CaseBranch, WhileStatement, RepeatStatement, ForStatement, Call, ParameterList, Variable, IndexComponent, FieldComponent, Literal, UnaryOp, BinaryOp, TypeRef, ArrayType, RangeType, EnumType, RecordType, dan FieldDecl.

ast.hpp

```

enum class AstKind {
    Program,
    DeclarationPart,
    ConstDecl,
    TypeDecl,
    VarDecl,
    ProcedureDecl,
    FunctionDecl,
    ParameterGroup,
    CompoundStatement,
    EmptyStatement,
    AssignStatement,
    IfStatement,
    CaseStatement,
    CaseBranch,
    WhileStatement,
    RepeatStatement,
    ForStatement,
    Call,
    ParameterList,
    Variable,
    IndexComponent,
    FieldComponent,
    Literal,
    UnaryOp,
    BinaryOp,
    TypeRef,
    ArrayType,
    RangeType,
    EnumType,
    RecordType,
    FieldDecl
};

enum class LiteralKind {
    None,
    Integer,
    Real,
    Char,
    String,
    Boolean,
    IdentifierConstant
};

struct AstAnnotation {
    int type_id = 0;
    int tab_index = -1;
    int block_index = 0;
    int lexical_level = 0;
    bool initialized = false;
};

struct AstNode;
using AstNodePtr =
std::shared_ptr<AstNode>;

struct AstNode {
    AstKind kind;
    SourceLocation location;
    std::string name;
    std::string value;
    std::string op;
    LiteralKind literal_kind =
LiteralKind::None;
    AstAnnotation annotation;
    std::vector<AstNodePtr> children;

    explicit AstNode(AstKind kind);
    AstNode(AstKind kind,
SourceLocation location);
    AstNodePtr add_child(AstNodePtr
child);
};

```

AstAnnotation menyimpan `type_id`, `tab_index`, `block_index`, `lexical_level`, dan `initialized`. Anotasi ini menjadi dasar Decorated AST yang dicetak setelah semantic analysis.

3. Integrasi Main Program dengan Tahap Semantic

Integrasi tahap semantic analysis berada pada `src/main.cpp`. Setelah parsing berhasil, program membuat objek `semantic::AstBuilder` dan memanggil `build(tree)`. Diagnostic dari AST builder dikumpulkan ke vector diagnostic utama. Jika root AST tersedia, program membuat `semantic::SemanticAnalyzer` dan memanggil `analyze(ast_result.root)`.

main.cpp

```

(int main(int argc, char* argv[]) {
    if (argc != 2) {
        cerr << "Usage: " << argv[0] <<
        " <input_file.txt>" << endl;
        return 1;
    }

    string input_file = argv[1];
    ifstream in(input_file);
    if (!in.is_open()) {
        cerr << "Failed to open input
file: " << input_file << endl;
        return 1;
    }
    in.close();

    Lexer lexer(input_file);
    vector<Token> tokens =
lexer.tokenize();

    bool has_lexer_error = false;
    for (const auto& token : tokens) {
        if (token.get_type() ==
TokenType::TOKEN_ERROR) {
            has_lexer_error = true;
            cerr << "Lexical error at
line " << token.get_line()
            << ", col " <<
token.get_column()
            << ": " <<
token.get_value() << endl;
        }
    }

    if (has_lexer_error) {
        cerr << "Parsing aborted due to
lexical errors." << endl;
        return 1;
    }

    vector<Token> filtered;
    for (const auto& token : tokens) {
        if (token.get_type() !=
TokenType::TOKEN_COMMENT &&
token.get_type() !=
TokenType::TOKEN_NEWLINE) {
            filtered.push_back(token);
        }
    }

    Parser parser(filtered);
    auto tree = parser.parse();

    const auto& errors =
parser.get_errors();
    cout << "=== Parse Tree ===" <<
endl;
    tree->print(cout);

    string filename = input_file;
    size_t last_slash =
filename.find_last_of("\\/");
    if (last_slash != string::npos) {
        filename =
filename.substr(last_slash + 1);
    }
    if (!errors.empty()) {
        cerr << endl << "Parser errors
(" << errors.size() << "):" << endl;
        for (const auto& err : errors)
        {
            cerr << "  " << err <<
endl;
        }
        return 1;
    }

    semantic::ASTBuilder ast_builder;
    auto ast_result =
ast_builder.Build(tree);

    semantic::SemanticResult
semantic_result;
    vector<semantic::Diagnostic>
diagnostics;
    append_diagnostics(diagnostics,
ast_result.diagnostics);

    if (ast_result.root) {
        semantic::SemanticAnalyzer
analyzer;
        semantic_result =
analyzer.analyze(ast_result.root);
        append_diagnostics(diagnostics,
semantic_result.diagnostics);
    }

    const auto report_root =
semantic_result.decorated_ast ?
semantic_result.decorated_ast :
ast_result.root;

    semantic::print_semantic_report(cout,
report_root,

    semantic_result.symbols,
    semantic_result.types,
    diagnostics);

    string semantic_output_path =
"test/alltestone-3/" + filename +
" SEMANTIC.txt";
    ofstream
semantic_out(semantic_output_path);
    if (semantic_out.is_open()) {
        semantic_out << "=== Parse Tree
===> << endl;
        tree->print(semantic_out);

        semantic::print_semantic_report(semantic_out,
report_root,

        semantic_result.symbols,
        semantic_result.types,
        diagnostics);
        semantic_out.close();
    } else {
        cerr << "Warning: Could not
open output file: " <<
semantic_output_path << endl;
    }

    if (semantic::has_errors(diagnostics)) {
        cerr << endl << "Semantic
errors:" << endl;
        print_diagnostic_summary(diagnostics);
        return 1;
    }

    return 0;
}

```

Hasil semantic analysis kemudian dicetak dengan `semantic::print_semantic_report()`. Program juga menulis output ke file `test/milestone-3/<nama_input>_SEMANTIC.txt`. Jika terdapat semantic error, program mencetak ringkasan error ke `stderr` dan mengembalikan exit code 1.

4. Implementasi Program dan Declaration Part

Pada tahap AST, top-level program direpresentasikan oleh node Program. Node ini menyimpan nama program dan dua anak utama, yaitu DeclarationPart dan CompoundStatement. Analyzer memulai penelusuran dari `visit_program()`. Nama program dimasukkan ke tab sebagai objek Program, kemudian declaration part dan compound statement dikunjungi berurutan.

semantic_analyzer.cpp
<pre>AstNodePtr SemanticAnalyzer::visit_program(const AstNodePtr& node) { if (!node) { semantic_error(nullptr, "AST root is empty"); return } make_ast(AstKind::Program); } const int program_index = add_symbol_checked(node, node->name, SymbolObject::Program, 0, 0, 1, true); node->annotation.tab_index = program_index; node->annotation.block_index = result->symbols.current_block_index(); node->annotation.lexical_level = result->symbols.current_level(); for (const auto& child : node- >children) { if (child && child->kind == AstKind::DeclarationPart) visit_declaration_part(child); else if (child && child->kind == AstKind::CompoundStatement) visit_compound_statement(child); } return node; } AstNodePtr SemanticAnalyzer::visit_declaration_par t(const AstNodePtr& node) { if (!node) return nullptr; for (const auto& child : node- >children) { if (!child) continue; switch (child->kind) { case AstKind::ConstDecl: visit_const_decl(child); break; case AstKind::TypeDecl: visit_type_decl(child); break; case AstKind::VarDecl: visit_var_decl(child); break; case AstKind::ProcedureDecl: case AstKind::FunctionDecl: visit_subprogram_decl(child); break; default: break; } } return node; }</pre>

Node CompoundStatement menyimpan daftar statement. Saat dikunjungi, analyzer memberi anotasi block_index dan lexical_level, lalu memanggil visit_statement() untuk setiap anak statement.

5. Implementasi Deklarasi, Simbol, Tabel, dan Type Resolution

Pada Milestone 3, deklarasi dianalisis oleh semantic_analyzer_declarations.cpp. visit_const_decl() mengevaluasi constant, menentukan tipe, memasukkan entry Constant ke tab, dan menandai konstanta sebagai initialized. visit_type_decl() menyelesaikan tipe dan memasukkannya sebagai objek Type. visit_var_decl() menyelesaikan tipe variable dan memasukkannya sebagai objek Variable dengan initialized=false.

Penyelesaian tipe dilakukan oleh resolve_type_node(). TypeRef harus mengacu pada identifier bertipe Type. RangeType memeriksa kesamaan tipe bound, melarang bound Real, dan memastikan lower bound tidak lebih besar dari upper bound. EnumType pada implementasi ini mengharuskan identifier enumerasi sudah dideklarasikan sebagai konstanta. ArrayType membuat entry atab, sedangkan RecordType membuat block baru pada btab dan memasukkan field sebagai objek Field.



Struktur symbol table yang digunakan adalah sebagai berikut.

Tabel	Atribut	Fungsi
tab	identifier, obj, type, ref, nrm, lev, adr, link, initialized, value	Menyimpan identifier dan metadata semantiknya.
btab	last, lpar, psze, vsze	Menyimpan metadata blok procedure, function, dan record.
atab	xtyp, etyp, eref, low, high, elsz, size	Menyimpan metadata array.

6. Implementasi Validasi Statement dan Subprogram

Statement dianalisis oleh `semantic_analyzer_statements.cpp`. `visit_assignment()` mengevaluasi target dan expression, mengecek apakah target assignable, lalu mengecek assignment compatibility. Jika valid, target ditandai initialized. `visit_if()`, `visit_while()`, dan `visit_repeat()` memastikan condition bertipe Boolean. `visit_for()` memastikan iterator adalah variable atau parameter yang assignable dan bertipe simple ordinal non-Real, kemudian mengecek bound awal dan akhir. `visit_case()` mengevaluasi selector dan memastikan setiap case label compatible dengan tipe selector. `visit_call()` memastikan identifier yang dipanggil merupakan procedure atau function, mengecek jumlah argument, dan mengecek tipe argument terhadap parameter formal.

Procedure dan function declaration dianalisis oleh `visit_subprogram_decl()`. Analyzer membuat block baru, memasukkan procedure atau function ke tab, melakukan `push_scope()`, memasukkan parameter sebagai objek Parameter, mengunjungi deklarasi lokal, mengunjungi body, lalu melakukan `pop_scope()`.

`semantic_analyzer_statements.cpp`

```

AstNodePtr
SemanticAnalyzer::visit_assignment(const
    AstNodePtr& node) {
    if (!node || node->children.size()
        < 2) return node;

    ValueInfo target =
    eval_variable(node->children[0], true);
    ValueInfo value =
    eval_expression(node->children[1]);

    node->annotation.type_id =
    target.type;
    node->annotation.tab_index =
    target.tab_index;

    bool assignable = false;
    std::string target_name = node-
    >children[0] ? node->children[0]->name
    : "<target>";
    if (target.base_tab_index >= 0 &&
        static_cast<size_t>
        (target.base_tab_index) < result-
        >symbols.tab_entries().size()) {
        const auto& entry = result-
        >symbols.tab_entry(target.base_tab_inde
        x);
        assignable =
        is_assignable_storage(target, true);
        if (!assignable) {
            semantic_error(node,
                "Identifier '" + target_name + "' is
                not an assignable target (" +
                symbol_object_name(entry.obj) + ")");
        }
    }

    const bool compatible = result-
    >types.assignment_compatible(
        target.type, value.type,
        value.is_constant
        ?
        std::optional<ConstantValue>
        (ConstantValue{value.type,
            value.string_value,

            value.int_value,
            value.real_value,

            value.char_value,
            value.bool_value})
        : std::nullopt);
    if (!compatible) {
        semantic_error(node, "Cannot
        assign " + result-
        >types.type_name(value.type) +
            " to " +
            result->types.type_name(target.type));
    }

    if (assignable && compatible &&
        value.type != 0) {
        mark_target_initialized(target);
    }

    return node;
}

```

7. Implementasi Validasi Expression dan Type Compability

Expression dianalisis oleh `semantic_analyzer_expressions.cpp`. Literal dikonversi menjadi `ValueInfo` oleh `value_from_literal()`. Variable dievaluasi oleh `eval_variable()`, termasuk lookup symbol, pengecekan apakah identifier merupakan value, pengecekan inisialisasi, array indexing, dan record field access. Expression umum dievaluasi oleh `eval_expression()`.

Operator unary not membutuhkan Boolean, sedangkan unary minus membutuhkan operand numeric. Operator and dan or membutuhkan Boolean. Operator div dan mod membutuhkan integer-like type. Operator aritmatika lain membutuhkan numeric type. Operator relasional menghasilkan Boolean dan membutuhkan operand yang compatible.

types.cpp

```
bool TypeRegistry::compatible(int
left_type, int right_type) const {
    if (left_type == 0 || right_type ==
0) return true;
    if (left_type == right_type) return
true;

    auto base = [&](int type_id) {
        if (type_id > 0 &&
static_cast<size_t>(type_id) <
types.size()) {
            const auto& type =
types[static_cast<size_t>(type_id)];
            if (type.kind ==
TypeKind::Subrange || type.kind ==
TypeKind::Enumerated) {
                return type.base_type;
            }
        }
        return type_id;
    };

    return base(left_type) ==
base(right_type);
}

bool
TypeRegistry::assignment_compatible(int
target_type, int value_type,
const std::optional<ConstantValue>&
value) const {
    if (target_type == 0 || value_type
== 0) return true;
    if (target_type == real_id &&
value_type == integer_id) return true;
    if (!compatible(target_type,
value_type)) return false;

    if (target_type > 0 &&
static_cast<size_t>(target_type) <
types.size() &&
types[static_cast<size_t>
(target_type)].kind ==
TypeKind::Subrange &&
value.has_value()) {
        const auto& type =
types[static_cast<size_t>
(target_type)];
        int scalar = 0;
        if (value->type_id ==
integer_id) scalar = value->int_value;
        else if (value->type_id ==
char_id) scalar = value->char_value;
        return scalar >= type.low &&
scalar <= type.high;
    }

    return true;
}
```

Aturan kompatibilitas tipe yang digunakan adalah sebagai berikut.

Aturan	Implementasi
Tipe sama	Compatible.
Subrange dan enumerated	Dibandingkan melalui base_type.
Assignment Real := Integer	Diterima.
Assignment ke subrange	Bound dicek bila nilai konstanta tersedia.
Aritmatika	Operand harus numeric.
div dan mod	Operand harus integer-like.
and, or, not	Operand harus Boolean.
Relational operator	Menghasilkan Boolean.
Array index	Harus simple non-Real type dan compatible dengan index type array.
Record field	Field harus terdaftar pada tipe record.

8. Diagnostics dan Output Semantic Report

Diagnostic semantik disimpan sebagai `semantic::Diagnostic`. Setiap semantic error memiliki severity, lokasi, dan pesan. Fungsi `semantic_error()` menambahkan prefix `Semantic error:` pada pesan. Program tetap mencetak Decorated AST dan Symbol Table meskipun semantic error ditemukan, selama parsing berhasil.

Validasi semantik yang dilakukan mencakup redeclaration dalam scope yang sama, identifier belum dideklarasikan, type belum dideklarasikan, target assignment tidak valid, assignment-incompatible, kondisi control flow bukan Boolean, iterator for tidak valid, case label tidak compatible, array indexing tidak valid, record field tidak ditemukan, call argument count/type tidak sesuai, penggunaan `readln` dan `writeln`, serta variable yang mungkin digunakan sebelum inisialisasi.

printer.cpp

```

void print_decorated_ast(std::ostream&
os, const AstNodePtr& root, const
TypeRegistry& types) {
    if (!root) {
        os << "<empty>\n";
        return;
    }

    print_node_label(os, *root, types);
    os << '\n';

    std::function<void(const
AstNodePtr&, const std::string&, bool)>
print_child =
    [&](const AstNodePtr& node,
const std::string& prefix, bool
is_last) {
        if (!node) return;
        os << prefix << (is_last ?
"\n-- " : " ");
        print_node_label(os, *node,
types);
        os << '\n';

        const std::string
next_prefix = prefix + (is_last ? "
" : " ");
        for (size_t i = 0; i <
node->children.size(); ++i) {
            print_child(node
->children[i], next_prefix, i + 1 ==
node->children.size());
        }

        for (size_t i = 0; i < root-
->children.size(); ++i) {
            print_child(root->children[i],
"", i + 1 == root->children.size());
        }
    };

    void print_symbol_tables(std::ostream&
os, const SymbolTable& symbols, const
TypeRegistry& types) {
        os << "==== Symbol Table: tab
====\n";
        os <<
"idx\tidentifier\tobj\ttype\tref\tname\t
lev\tadr\tlink\tinit\tvalue\n";
        const auto& tab =
symbols.tab_entries();
        for (size_t i = 0; i < tab.size();
++i) {
            const auto& entry = tab[i];
            os << i << '\t' <<
entry.identifier << '\t' <<
symbol_object_name(entry.obj) << '\t'
<<
entry.type_name(entry.type) << '\t' <<
entry.ref << '\t' << entry.name << '\t'
<< entry.lev << '\t' <<
entry.adr << '\t' << entry.link << '\t'
<< (entry.initialized ?
"yes" : "no") << '\t' << entry.value <<
'\n';
        }

        os << "==== Symbol Table: btab
====\n";
        os <<
"idx\tlast\tlpar\tlpsize\tvsize\n";
        const auto& btab =
symbols.btab_entries();
        for (size_t i = 0; i < btab.size();
++i) {
            os << i << '\t' << btab[i].last
<< '\t' << btab[i].lpar << '\t'
<< btab[i].lpsize << '\t' <<
btab[i].vsize << '\n';
        }

        os << "==== Symbol Table: atab
====\n";
        os <<
"idx\txtype\tetype\teref\tlow\thigh\telsz
\tsize\n";
        const auto& atab =
symbols.atab_entries();
        for (size_t i = 0; i < atab.size();
++i) {
            os << i << '\t' <<
types.type_name(atab[i].xtype) << '\t'
<<
types.type_name(atab[i].etype) << '\t'
<< atab[i].eref << '\t'
<< atab[i].low << '\t' <<
atab[i].high << '\t'
<< atab[i].elsz << '\t' <<
atab[i].size << '\n';
        }
    };

    void print_diagnostics(std::ostream&
os, const std::vector<Diagnostic>&
diagnostics) {
        os << "==== Semantic Errors
====\n";
        if (diagnostics.empty()) {
            os << "No semantic errors.\n";
            return;
        }

        for (const auto& diagnostic :
diagnostics) {
            if (diagnostic.location.line >
0 || diagnostic.location.column > 0) {
                os <<
diagnostic.location.line << " " <<
diagnostic.location.column << " ";
            }
            os <<
severity_name(diagnostic.severity) <<
": " << diagnostic.message << '\n';
        }
    };

    void
print_semantic_report(std::ostream& os,
const AstNodePtr& root,
const SymbolTable& symbols,
const TypeRegistry& types,
const std::vector<Diagnostic>& diagnostics) {
        os << "==== Decorated AST
====\n";
        print_decorated_ast(os, root,
types);
        print_symbol_tables(os, symbols,
types);
        print_diagnostics(os, diagnostics);
    };
}

```


Program mencetak output berikut.

Bagian output	Isi
Parse Tree	Parse tree hasil parser.
Decorated AST	AST yang telah diberi anotasi tipe, index symbol table, lexical level, dan block index.
Symbol Table: tab	Tabel identifier utama.
Symbol Table: btabs	Tabel blok.
Symbol Table: atab	Tabel array.
Semantic Errors	Daftar diagnostic semantik atau No semantic errors.

III. Pengujian

Test Case / Input / Output
Test Case 1 : tes struktur program valid yang paling dasar, komentar yang tidak menjadi node parse tree, empty statement, deklarasi variabel integer, dan assignment sederhana
<pre>test > milestone-3 >  tc1.txt 1 program BasicValid; 2 { Valid: empty statement, comments ignored, basic integer assignment. } 3 var 4 x: integer; 5 begin 6 ; 7 x := 1; 8 end. 9 </pre>

```

test > milestone-3 > tc1.txt_SEMANTIC.txt
34   └─ period
35
36   === Decorated AST ===
37   Program(BasicValid, tab_index=41
38   |-- DeclarationPart
39   |   \-- VarDecl(x) : type=Integer, tab_index=42
40   |       \-- TypeRef(integer) : type=Integer, tab_index=32
41   |-- CompoundStatement
42   |   |-- EmptyStatement
43   |   |-- AssignStatement : type=Integer, tab_index=42
44   |   |   |-- Variable(x) : type=Integer, tab_index=42
45   |   |   \-- Literal(1) value=1 literal=Integer : type=Integer
46   |   \-- EmptyStatement
47
48   === Symbol Table: tab ===
49   idx identifier  obj type    ref nrm lev adr link  init  value
50   0  and reserved  unknown 0  1  0  0  0  yes
51   1  array reserved  unknown 0  1  0  0  0  yes
52   2  begin reserved  unknown 0  1  0  0  1  yes
53   3  case reserved  unknown 0  1  0  0  2  yes
54   4  const reserved  unknown 0  1  0  0  3  yes
55   5  div reserved   unknown 0  1  0  0  4  yes
56   6  downto reserved unknown 0  1  0  0  5  yes
57   7  do reserved   unknown 0  1  0  0  6  yes
58   8  else reserved  unknown 0  1  0  0  7  yes
59   9  end reserved   unknown 0  1  0  0  8  yes
60  10  for reserved   unknown 0  1  0  0  9  yes
61  11  function reserved unknown 0  1  0  0 10  yes
62  12  if reserved   unknown 0  1  0  0 11  yes
63  13  mod reserved   unknown 0  1  0  0 12  yes
64  14  not reserved   unknown 0  1  0  0 13  yes
65  15  of reserved   unknown 0  1  0  0 14  yes
66  16  or reserved   unknown 0  1  0  0 15  yes
67  17  procedure reserved unknown 0  1  0  0 16  yes
68  18  program reserved unknown 0  1  0  0 17  yes
69  19  record reserved unknown 0  1  0  0 18  yes
70  20  repeat reserved unknown 0  1  0  0 19  yes
71  21  integer reserved unknown 0  1  0  0 20  yes
72  22  real reserved  unknown 0  1  0  0 21  yes
73  23  boolean reserved unknown 0  1  0  0 22  yes
74  24  char reserved  unknown 0  1  0  0 23  yes
75  25  string reserved unknown 0  1  0  0 24  yes
76  26  then reserved  unknown 0  1  0  0 25  yes
77  27  to reserved   unknown 0  1  0  0 26  yes
78  28  type reserved  unknown 0  1  0  0 27  yes
79  29  until reserved  unknown 0  1  0  0 28  yes
80  30  var reserved   unknown 0  1  0  0 29  yes
81  31  while reserved  unknown 0  1  0  0 30  yes
82  32  Integer type   Integer 0  1  0  0 31  yes
83  33  Real type      Real    0  1  0  0 32  yes
84  34  Char type      Char    0  1  0  0 33  yes
85  35  Boolean type   Boolean  0  1  0  0 34  yes
86  36  String type    String   0  1  0  0 35  yes
87  37  True constant  Boolean  0  1  0  1 36  yes true
88  38  False constant Boolean  0  1  0  0 37  yes false
89  39  readln procedure unknown 0  1  0  0 38  yes
90  40  writeln procedure unknown 0  1  0  0 39  yes
91  41  BasicValid program unknown 0  1  0  0 40  yes
92  42  x variable    Integer 0  1  0  0 41  yes
93
94   === Symbol Table: btab ===
95   idx last  lpar  psze  vsze
96   0  42  0  0  1
97
98   === Symbol Table: atab ===
99   idx xtyp  etyp  eref  low high  elsz  size
100
101   === Semantic Errors ===
102   No semantic errors.
103

```

Test Case 2: tes kemampuan semantic analyzer dalam menginisialisasi predefined identifier (Boolean, true, false), memproses deklarasi const, type, dan var, serta membentuk symbol table untuk subrange, enumerated type, array, dan record

```
test > milestone-3 > tc2.txt
1  program PredefinedAndDeclarationsValid;
2  const
3      Low == 1;
4      High == 5;
5      Red == 1;
6      Green == 2;
7      Blue == 3;
8  type
9      SmallRange == Low..High;
10     ColorCode == (Red, Green, Blue);
11     IntVector == array[SmallRange] of integer;
12     Student == record
13         id: integer;
14         grade: char;
15         scores: IntVector;
16     end;
17 var
18     flag, other: Boolean;
19     index: SmallRange;
20     color: ColorCode;
21     student: Student;
22 begin
23     flag := true;
24     other := FALSE;
25     index := 3;
26     color := Red;
27     student.id := index;
28     student.grade := 'A';
29     student.scores[index] := 100;
30
31     if flag and not other or false then
32     begin
33         other := true;
34     end
35     else
36     begin
37         other := false;
38     end;
39 end.
```

```

test > milestone-3 > tc2.txt_SEMANTIC.txt
296
297 === Decorated AST ===
298 Program(PredefinedAndDeclarationsValid), tab_index=41
299 |-- DeclarationPart
300 | |-- ConstDecl(Low) : type=Integer, tab_index=42
301 | | \-- Literal(1) value=1 literal=Integer : type=Integer
302 | |-- ConstDecl(High) : type=Integer, tab_index=43
303 | | \-- Literal(5) value=5 literal=Integer : type=Integer
304 | |-- ConstDecl(Red) : type=Integer, tab_index=44
305 | | \-- Literal(1) value=1 literal=Integer : type=Integer
306 | |-- ConstDecl(Green) : type=Integer, tab_index=45
307 | | \-- Literal(2) value=2 literal=Integer : type=Integer
308 | |-- ConstDecl(Blue) : type=Integer, tab_index=46
309 | | \-- Literal(3) value=3 literal=Integer : type=Integer
310 | |-- TypeDecl(SmallRange) : type=SmallRange, tab_index=47
311 | | \-- RangeType : type=SmallRange
312 | | |-- Literal(Low) value=Low literal=IdentifierConstant : type=Integer, tab_index=42
313 | | \-- Literal(High) value=High literal=IdentifierConstant : type=Integer, tab_index=43
314 | |-- TypeDecl(ColorCode) : type=ColorCode, tab_index=48
315 | | \-- EnumType : type=ColorCode
316 | | |-- Literal(Red) value=Red literal=IdentifierConstant
317 | | |-- Literal(Green) value=Green literal=IdentifierConstant
318 | | |-- Literal(Blue) value=Blue literal=IdentifierConstant
319 | |-- TypeDecl(IntVector) : type=IntVector, tab_index=49
320 | | \-- ArrayType : type=IntVector
321 | | |-- TypeRef(SmallRange) : type=SmallRange, tab_index=47
322 | | |-- TypeRef(integer) : type=Integer, tab_index=32
323 | |-- TypeDecl(Student) : type=Student, tab_index=53
324 | | \-- RecordType : type=Student
325 | | |-- FieldDecl(id) : type=Integer, tab_index=50
326 | | | |-- TypeRef(integer) : type=Integer, tab_index=32
327 | | |-- FieldDecl(grade) : type=Char, tab_index=51
328 | | | |-- TypeRef(char) : type=Char, tab_index=34
329 | | |-- FieldDecl(scores) : type=IntVector, tab_index=52
330 | | | |-- TypeRef(IntVector) : type=IntVector, tab_index=49
331 | |-- VarDecl(flag) : type=Boolean, tab_index=54
332 | | |-- TypeRef(Boolean) : type=Boolean, tab_index=35
333 | |-- VarDecl(other) : type=Boolean, tab_index=55
334 | | |-- TypeRef(Boolean) : type=Boolean, tab_index=35
335 | |-- VarDecl(index) : type=SmallRange, tab_index=56
336 | | |-- TypeRef(SmallRange) : type=SmallRange, tab_index=47
337 | |-- VarDecl(color) : type=ColorCode, tab_index=57
338 | | |-- TypeRef(ColorCode) : type=ColorCode, tab_index=48
339 | |-- VarDecl(student) : type=Student, tab_index=58
340 | | |-- TypeRef(Student) : type=Student, tab_index=53
341 | \-- CompoundStatement
342 | |-- AssignStatement : type=Boolean, tab_index=54
343 | | |-- Variable(flag) : type=Boolean, tab_index=54
344 | | |-- Variable(true) : type=Boolean, tab_index=37
345 | |-- AssignStatement : type=Boolean, tab_index=55
346 | | |-- Variable(other) : type=Boolean, tab_index=55
347 | | |-- Variable(FALSE) : type=Boolean, tab_index=38
348 | |-- AssignStatement : type=SmallRange, tab_index=56
349 | | |-- Variable(index) : type=SmallRange, tab_index=56
350 | | |-- Literal(3) value=3 literal=Integer : type=Integer
351 | |-- AssignStatement : type=ColorCode, tab_index=57
352 | | |-- Variable(color) : type=ColorCode, tab_index=57
353 | | |-- Variable(Red) : type=Integer, tab_index=44
354 | |-- AssignStatement : type=Integer, tab_index=50
355 | | |-- Variable(student) : type=Integer, tab_index=50
356 | | | |-- FieldComponent(id)
357 | | |-- Variable(index) : type=SmallRange, tab_index=56
358 | |-- AssignStatement : type=Char, tab_index=51
359 | | |-- Variable(student) : type=Char, tab_index=51
360 | | | |-- FieldComponent(grade)
361 | | |-- Literal('A') value='A' literal=Char : type=Char
362 | |-- AssignStatement : type=Integer, tab_index=52
363 | | |-- Variable(student) : type=Integer, tab_index=52
364 | | | |-- FieldComponent(scores)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

=== Symbol Table: tab ===

idx	identifier	obj type	ref	nrm	lev	adr	link	init	value
0	and reserved	unknown	0	1	0	0	0	yes	
1	array reserved	unknown	0	1	0	0	0	yes	
2	begin reserved	unknown	0	1	0	0	1	yes	
3	case reserved	unknown	0	1	0	0	2	yes	
4	const reserved	unknown	0	1	0	0	3	yes	
5	div reserved	unknown	0	1	0	0	4	yes	
6	downto reserved	unknown	0	1	0	0	5	yes	
7	do reserved	unknown	0	1	0	0	6	yes	
8	else reserved	unknown	0	1	0	0	7	yes	
9	end reserved	unknown	0	1	0	0	8	yes	
10	for reserved	unknown	0	1	0	0	9	yes	
11	function reserved	unknown	0	1	0	0	10	yes	
12	if reserved	unknown	0	1	0	0	11	yes	
13	mod reserved	unknown	0	1	0	0	12	yes	
14	not reserved	unknown	0	1	0	0	13	yes	
15	of reserved	unknown	0	1	0	0	14	yes	
16	or reserved	unknown	0	1	0	0	15	yes	
17	procedure reserved	unknown	0	1	0	0	16	yes	
18	program reserved	unknown	0	1	0	0	17	yes	
19	record reserved	unknown	0	1	0	0	18	yes	
20	repeat reserved	unknown	0	1	0	0	19	yes	
21	integer reserved	unknown	0	1	0	0	20	yes	
22	real reserved	unknown	0	1	0	0	21	yes	
23	boolean reserved	unknown	0	1	0	0	22	yes	
24	char reserved	unknown	0	1	0	0	23	yes	
25	string reserved	unknown	0	1	0	0	24	yes	
26	then reserved	unknown	0	1	0	0	25	yes	
27	to reserved	unknown	0	1	0	0	26	yes	
28	type reserved	unknown	0	1	0	0	27	yes	
29	until reserved	unknown	0	1	0	0	28	yes	
30	var reserved	unknown	0	1	0	0	29	yes	
31	while reserved	unknown	0	1	0	0	30	yes	
32	Integer type	Integer	0	1	0	0	31	yes	
33	Real type	Real	0	1	0	0	32	yes	
34	Char type	Char	0	1	0	0	33	yes	
35	Boolean type	Boolean	0	1	0	0	34	yes	
36	String type	String	0	1	0	0	35	yes	
37	True constant	Boolean	0	1	0	1	36	yes	true
38	False constant	Boolean	0	1	0	0	37	yes	false
39	readln procedure	unknown	0	1	0	0	38	yes	
40	writeln procedure	unknown	0	1	0	0	39	yes	
41	PredefinedAndDeclarationsValid	program	unknown	0	1	0	0	40	yes
42	Low constant	Integer	0	1	0	0	41	yes	1
43	High constant	Integer	0	1	0	0	42	yes	5
44	Red constant	Integer	0	1	0	0	43	yes	1
45	Green constant	Integer	0	1	0	0	44	yes	2
46	Blue constant	Integer	0	1	0	0	45	yes	3
47	SmallRange type	SmallRange	0	1	0	0	46	yes	
48	ColorCode type	ColorCode	0	1	0	0	47	yes	
49	IntVector type	IntVector	0	1	0	0	48	yes	
50	id field	Integer	0	1	0	0	0	yes	
51	grade field	Char	0	1	1	1	50	yes	
52	scores field	IntVector	0	1	1	2	51	yes	
53	Student type	Student	1	1	0	0	49	yes	
54	flag variable	Boolean	0	1	0	0	53	yes	
55	other variable	Boolean	0	1	0	1	54	yes	
56	index variable	SmallRange	0	1	0	2	55	yes	
57	color variable	ColorCode	0	1	0	3	56	yes	
58	student variable	Student	1	1	0	4	57	yes	

Test Case 3: tes deklarasi prosedur dan fungsi, parameter formal, scope lokal dan global, pemanggilan prosedur, pemanggilan fungsi rekursif, dan assignment ke nama fungsi sebagai nilai balik

```
test > milestone-3 > tc3.txt
1  program ScopeFunctionValid;
2  var
3  |   globalValue: integer;
4
5  procedure AddToGlobal(delta: integer);
6  var
7  |   localValue: integer;
8  begin
9  |   localValue := delta;
10 |   globalValue := globalValue + localValue;
11 end;
12
13 function Factorial(n: integer): integer;
14 begin
15 |   if n == 0 then
16 |   begin
17 |   |   Factorial := 1;
18 |   end
19 |   else
20 |   begin
21 |   |   Factorial := n * Factorial(n - 1);
22 |   end;
23 end;
24
25 begin
26 |   globalValue := 1;
27 |   AddToGlobal(2);
28 |   globalValue := Factorial(5);
29 end.
30
```

```

=== Symbol Table: tab ===
idx identifier obj type ref nrm lev adr link init value
0 and reserved unknown 0 1 0 0 0 0 yes
1 array reserved unknown 0 1 0 0 0 0 yes
2 begin reserved unknown 0 1 0 0 1 1 yes
3 case reserved unknown 0 1 0 0 2 2 yes
4 const reserved unknown 0 1 0 0 3 3 yes
5 div reserved unknown 0 1 0 0 4 4 yes
6 downto reserved unknown 0 1 0 0 5 5 yes
7 do reserved unknown 0 1 0 0 6 6 yes
8 else reserved unknown 0 1 0 0 7 7 yes
9 end reserved unknown 0 1 0 0 8 8 yes
10 for reserved unknown 0 1 0 0 9 9 yes
11 function reserved unknown 0 1 0 0 10 10 yes
12 if reserved unknown 0 1 0 0 11 11 yes
13 mod reserved unknown 0 1 0 0 12 12 yes
14 not reserved unknown 0 1 0 0 13 13 yes
15 of reserved unknown 0 1 0 0 14 14 yes
16 or reserved unknown 0 1 0 0 15 15 yes
17 procedure reserved unknown 0 1 0 0 16 16 yes
18 program reserved unknown 0 1 0 0 17 17 yes
19 record reserved unknown 0 1 0 0 18 18 yes
20 repeat reserved unknown 0 1 0 0 19 19 yes
21 integer reserved unknown 0 1 0 0 20 20 yes
22 real reserved unknown 0 1 0 0 21 21 yes
23 boolean reserved unknown 0 1 0 0 22 22 yes
24 char reserved unknown 0 1 0 0 23 23 yes
25 string reserved unknown 0 1 0 0 24 24 yes
26 then reserved unknown 0 1 0 0 25 25 yes
27 to reserved unknown 0 1 0 0 26 26 yes
28 type reserved unknown 0 1 0 0 27 27 yes
29 until reserved unknown 0 1 0 0 28 28 yes
30 var reserved unknown 0 1 0 0 29 29 yes
31 while reserved unknown 0 1 0 0 30 30 yes
32 Integer type Integer 0 1 0 0 31 31 yes
33 Real type Real 0 1 0 0 32 32 yes
34 Char type Char 0 1 0 0 33 33 yes
35 Boolean type Boolean 0 1 0 0 34 34 yes
36 String type String 0 1 0 0 35 35 yes
37 True constant Boolean 0 1 0 1 36 36 yes true
38 False constant Boolean 0 1 0 0 37 37 yes false
39 readln procedure unknown 0 1 0 0 38 38 yes
40 writeln procedure unknown 0 1 0 0 39 39 yes
41 ScopeFunctionValid program unknown 0 1 0 0 40 40 yes
42 globalValue variable Integer 0 1 0 0 41 41 yes
43 AddToGlobal procedure unknown 1 1 0 1 42 42 yes
44 delta parameter Integer 0 1 1 0 0 43 43 yes
45 localValue variable Integer 0 1 1 1 44 44 yes
46 Factorial function Integer 2 1 0 1 43 43 yes
47 n parameter Integer 0 1 1 0 0 44 44 yes

=== Symbol Table: btab ===
idx last lpar psze vsze
0 46 0 0 1
1 45 44 1 1
2 47 47 1 0

=== Symbol Table: atab ===
idx xtyp etyp eref low high elsz size

=== Semantic Errors ===
No semantic errors.

```


Test Case 4: tes inferensi tipe pada ekspresi aritmatika dan boolean yang kompleks, termasuk prioritas operator, operator div, mod, /, relasional, and, or, dan not

```
test > milestone-3 > tc4.txt
1  program ExpressionValid;
2  var
3    i, j, k: integer;
4    r: real;
5    b: boolean;
6  begin
7    i := 2;
8    j := 3;
9    k := i + j * 4 - (i div 2) + (j mod 2);
10   r := i / j + 3.5;
11   b := (k >= 10) and (r > 0.0) or not false;
12 end.
13 |
```

```

test > milestone-3 > tc4.txt_SEMANTIC.txt
180 | | | | | ident(false)
181 | | | | | semicolon
182 | | | | | <statement>
183 | | | | | endsy
184 | | | | | period
185 |
186 === Decorated AST ===
187 Program(ExpressionValid), tab_index=41
188 |-- DeclarationPart
189 | |-- VarDecl(i) : type=Integer, tab_index=42
190 | | |-- TypeRef(integer) : type=Integer, tab_index=32
191 | |-- VarDecl(j) : type=Integer, tab_index=43
192 | | |-- TypeRef(integer) : type=Integer, tab_index=32
193 | |-- VarDecl(k) : type=Integer, tab_index=44
194 | | |-- TypeRef(integer) : type=Integer, tab_index=32
195 | |-- VarDecl(r) : type=Real, tab_index=45
196 | | |-- TypeRef(real) : type=Real, tab_index=33
197 | |-- VarDecl(b) : type=Boolean, tab_index=46
198 | | |-- TypeRef(boolean) : type=Boolean, tab_index=35
199 |-- CompoundStatement
200 | |-- AssignStatement : type=Integer, tab_index=42
201 | | |-- Variable(i) : type=Integer, tab_index=42
202 | | | |-- Literal(2) value=2 literal=Integer : type=Integer
203 | |-- AssignStatement : type=Integer, tab_index=43
204 | | |-- Variable(j) : type=Integer, tab_index=43
205 | | | |-- Literal(3) value=3 literal=Integer : type=Integer
206 | |-- AssignStatement : type=Integer, tab_index=44
207 | | |-- Variable(k) : type=Integer, tab_index=44
208 | | |-- BinaryOp op=plus : type=Integer
209 | | | |-- BinaryOp op=minus : type=Integer
210 | | | | |-- BinaryOp op=plus : type=Integer
211 | | | | | |-- Variable(i) : type=Integer, tab_index=42
212 | | | | | |-- BinaryOp op=times : type=Integer
213 | | | | | | |-- Variable(j) : type=Integer, tab_index=43
214 | | | | | | |-- Literal(4) value=4 literal=Integer : type=Integer
215 | | | | |-- BinaryOp op=idiv : type=Integer
216 | | | | | |-- Variable(i) : type=Integer, tab_index=42
217 | | | | | |-- Literal(2) value=2 literal=Integer : type=Integer
218 | | | |-- BinaryOp op=imod : type=Integer
219 | | | | |-- Variable(j) : type=Integer, tab_index=43
220 | | | | |-- Literal(2) value=2 literal=Integer : type=Integer
221 | |-- AssignStatement : type=Real, tab_index=45
222 | | |-- Variable(r) : type=Real, tab_index=45
223 | | |-- BinaryOp op=plus : type=Real
224 | | | |-- BinaryOp op=rdiv : type=Real
225 | | | | |-- Variable(i) : type=Integer, tab_index=42
226 | | | | |-- Variable(j) : type=Integer, tab_index=43
227 | | | | |-- Literal(3.5) value=3.5 literal=Real : type=Real
228 | |-- AssignStatement : type=Boolean, tab_index=46
229 | | |-- Variable(b) : type=Boolean, tab_index=46
230 | | |-- BinaryOp op=orsy : type=Boolean
231 | | | |-- BinaryOp op=andsy : type=Boolean
232 | | | | |-- BinaryOp op=geq : type=Boolean
233 | | | | | |-- Variable(k) : type=Integer, tab_index=44
234 | | | | | |-- Literal(10) value=10 literal=Integer : type=Integer
235 | | | | |-- BinaryOp op=gtr : type=Boolean
236 | | | | | |-- Variable(r) : type=Real, tab_index=45
237 | | | | | |-- Literal(0.0) value=0.0 literal=Real : type=Real
238 | | | |-- UnaryOp op=notsy : type=Boolean, tab_index=38
239 | | | | |-- Variable(false) : type=Boolean, tab_index=38
240 |-- EmptyStatement
241

```

```

=== Symbol Table: tab ===
idx identifier obj type ref nrm lev adr link init value
0 and reserved unknown 0 1 0 0 0 yes
1 array reserved unknown 0 1 0 0 0 yes
2 begin reserved unknown 0 1 0 0 1 yes
3 case reserved unknown 0 1 0 0 2 yes
4 const reserved unknown 0 1 0 0 3 yes
5 div reserved unknown 0 1 0 0 4 yes
6 downto reserved unknown 0 1 0 0 5 yes
7 do reserved unknown 0 1 0 0 6 yes
8 else reserved unknown 0 1 0 0 7 yes
9 end reserved unknown 0 1 0 0 8 yes
10 for reserved unknown 0 1 0 0 9 yes
11 function reserved unknown 0 1 0 0 10 yes
12 if reserved unknown 0 1 0 0 11 yes
13 mod reserved unknown 0 1 0 0 12 yes
14 not reserved unknown 0 1 0 0 13 yes
15 of reserved unknown 0 1 0 0 14 yes
16 or reserved unknown 0 1 0 0 15 yes
17 procedure reserved unknown 0 1 0 0 16 yes
18 program reserved unknown 0 1 0 0 17 yes
19 record reserved unknown 0 1 0 0 18 yes
20 repeat reserved unknown 0 1 0 0 19 yes
21 integer reserved unknown 0 1 0 0 20 yes
22 real reserved unknown 0 1 0 0 21 yes
23 boolean reserved unknown 0 1 0 0 22 yes
24 char reserved unknown 0 1 0 0 23 yes
25 string reserved unknown 0 1 0 0 24 yes
26 then reserved unknown 0 1 0 0 25 yes
27 to reserved unknown 0 1 0 0 26 yes
28 type reserved unknown 0 1 0 0 27 yes
29 until reserved unknown 0 1 0 0 28 yes
30 var reserved unknown 0 1 0 0 29 yes
31 while reserved unknown 0 1 0 0 30 yes
32 Integer type Integer 0 1 0 0 31 yes
33 Real type Real 0 1 0 0 32 yes
34 Char type Char 0 1 0 0 33 yes
35 Boolean type Boolean 0 1 0 0 34 yes
36 String type String 0 1 0 0 35 yes
37 True constant Boolean 0 1 0 1 36 yes true
38 False constant Boolean 0 1 0 0 37 yes false
39 readln procedure unknown 0 1 0 0 38 yes
40 writeln procedure unknown 0 1 0 0 39 yes
41 ExpressionValid program unknown 0 1 0 0 40 yes
42 i variable Integer 0 1 0 0 41 yes
43 j variable Integer 0 1 0 1 42 yes
44 k variable Integer 0 1 0 2 43 yes
45 r variable Real 0 1 0 3 44 yes
46 b variable Boolean 0 1 0 5 45 yes

=== Symbol Table: btab ===
idx last lpar psze vsze
0 46 0 0 6

=== Symbol Table: atab ===
idx xtyp etyp eref low high elsz size

=== Semantic Errors ===
No semantic errors.

```

Test Case 5: tes semantic control flow pada if, case, while, repeat-until, dan for, termasuk aturan Milestone 3 bahwa body while dan for harus berupa compound statement

```
test > milestone-3 > tc5.txt
```

```
1  program ControlFlowValid;
2  var
3      i, total: integer;
4      done: boolean;
5      grade: char;
6  begin
7      total := 0;
8      done := false;
9      grade := 'B';
10
11     for i := 1 to 3 do
12     begin
13         total := total + i;
14     end;
15
16     while total < 10 do
17     begin
18         total := total + 1;
19     end;
20
21     repeat
22     |   total := total - 1;
23     until total == 6;
24
25     case grade of
26     |   'A': begin total := total + 4; end;
27     |   'B', 'C': begin total := total + 2; end;
28     |   'D': begin total := total + 1; end;
29     end;
30
31     if total >= 0 then
32     begin
33         done := true;
34     end;
35 end.
36
```

```

320
321 === Decorated AST ===
322 Program(ControlFlowValid), tab_index=41
323 |-- DeclarationPart
324 | |-- VarDecl(i) : type=Integer, tab_index=42
325 | | \-- TypeRef(integer) : type=Integer, tab_index=32
326 | |-- VarDecl(total) : type=Integer, tab_index=43
327 | | \-- TypeRef(integer) : type=Integer, tab_index=32
328 | |-- VarDecl(done) : type=Boolean, tab_index=44
329 | | \-- TypeRef(boolean) : type=Boolean, tab_index=35
330 | | \-- VarDecl(grade) : type=Char, tab_index=45
331 | | \-- TypeRef(char) : type=Char, tab_index=34
332 | \-- CompoundStatement
333 | |-- AssignStatement : type=Integer, tab_index=43
334 | | |-- Variable(total) : type=Integer, tab_index=43
335 | | \-- Literal(0) value=0 literal=Integer : type=Integer
336 | |-- AssignStatement : type=Boolean, tab_index=44
337 | | |-- Variable(done) : type=Boolean, tab_index=44
338 | | \-- Variable(false) : type=Boolean, tab_index=38
339 | |-- AssignStatement : type=Char, tab_index=45
340 | | |-- Variable(grade) : type=Char, tab_index=45
341 | | \-- Literal('B') value='B' literal=Char : type=Char
342 | |-- ForStatement(i) op=tosy, tab_index=42
343 | | |-- Literal(1) value=1 literal=Integer : type=Integer
344 | | |-- Literal(3) value=3 literal=Integer : type=Integer
345 | | \-- CompoundStatement
346 | | |-- AssignStatement : type=Integer, tab_index=43
347 | | | |-- Variable(total) : type=Integer, tab_index=43
348 | | | \-- BinaryOp op=plus : type=Integer
349 | | | |-- Variable(total) : type=Integer, tab_index=43
350 | | | \-- Variable(i) : type=Integer, tab_index=42
351 | | \-- EmptyStatement
352 | |-- WhileStatement : type=Boolean
353 | | |-- BinaryOp op=lss : type=Boolean
354 | | | |-- Variable(total) : type=Integer, tab_index=43
355 | | | \-- Literal(10) value=10 literal=Integer : type=Integer
356 | | \-- CompoundStatement
357 | | |-- AssignStatement : type=Integer, tab_index=43
358 | | | |-- Variable(total) : type=Integer, tab_index=43
359 | | | \-- BinaryOp op=plus : type=Integer
360 | | | |-- Variable(total) : type=Integer, tab_index=43
361 | | | \-- Literal(1) value=1 literal=Integer : type=Integer
362 | | \-- EmptyStatement
363 | |-- RepeatStatement : type=Boolean
364 | | |-- CompoundStatement
365 | | | |-- AssignStatement : type=Integer, tab_index=43
366 | | | | |-- Variable(total) : type=Integer, tab_index=43
367 | | | | \-- BinaryOp op=minus : type=Integer
368 | | | | |-- Variable(total) : type=Integer, tab_index=43
369 | | | | \-- Literal(1) value=1 literal=Integer : type=Integer
370 | | | \-- EmptyStatement
371 | | \-- BinaryOp op=eql : type=Boolean
372 | | |-- Variable(total) : type=Integer, tab_index=43
373 | | \-- Literal(6) value=6 literal=Integer : type=Integer
374 | |-- CaseStatement : type=Char
375 | | |-- Variable(grade) : type=Char, tab_index=45
376 | | |-- CaseBranch
377 | | | |-- Literal('A') value='A' literal=Char : type=Char
378 | | | \-- CompoundStatement
379 | | | |-- AssignStatement : type=Integer, tab_index=43
380 | | | | |-- Variable(total) : type=Integer, tab_index=43
381 | | | | \-- BinaryOp op=plus : type=Integer
382 | | | | |-- Variable(total) : type=Integer, tab_index=43
383 | | | | \-- Literal(4) value=4 literal=Integer : type=Integer
384 | | | \-- EmptyStatement
385 | | |-- CaseBranch

```

```

384 | | | \-- EmptyStatement
385 | | | \-- CaseBranch
386 | | | |-- Literal('B') value='B' literal=Char : type=Char
387 | | | |-- Literal('C') value='C' literal=Char : type=Char
388 | | | \-- CompoundStatement
389 | | | |-- AssignStatement : type=Integer, tab_index=43
390 | | | | |-- Variable(total) : type=Integer, tab_index=43
391 | | | | \-- BinaryOp op=plus : type=Integer
392 | | | | |-- Variable(total) : type=Integer, tab_index=43
393 | | | | \-- Literal(2) value=2 literal=Integer : type=Integer
394 | | | \-- EmptyStatement
395 | | \-- CaseBranch
396 | | |-- Literal('D') value='D' literal=Char : type=Char
397 | | \-- CompoundStatement
398 | | |-- AssignStatement : type=Integer, tab_index=43
399 | | | |-- Variable(total) : type=Integer, tab_index=43
400 | | | \-- BinaryOp op=plus : type=Integer
401 | | | |-- Variable(total) : type=Integer, tab_index=43
402 | | | | \-- Literal(1) value=1 literal=Integer : type=Integer
403 | | \-- EmptyStatement
404 | \-- IfStatement : type=Boolean
405 | |-- BinaryOp op=geq : type=Boolean
406 | | |-- Variable(total) : type=Integer, tab_index=43
407 | | |-- Literal(0) value=0 literal=Integer : type=Integer
408 | | \-- CompoundStatement
409 | | |-- AssignStatement : type=Boolean, tab_index=44
410 | | | |-- Variable(done) : type=Boolean, tab_index=44
411 | | | |-- Variable(true) : type=Boolean, tab_index=37
412 | | \-- EmptyStatement
413 | \-- EmptyStatement
414

```

Test Case 6: tes penggunaan structured type yang kompleks, yaitu array berindeks subrange, record dengan field array, array of record, nested loop, akses field/index berantai, dan assignment antar tipe komposit yang sama

```

test > milestone-3 > tc6.txt
1  program ArrayRecordValid;
2  type
3      ScoreIndex == 1..3;
4      ScoreArray == array[ScoreIndex] of integer;
5      Student == record
6          id: integer;
7          scores: ScoreArray;
8      end;
9      Classroom == array[1..2] of Student;
10 var
11     classes: Classroom;
12     firstStudent, copiedStudent: Student;
13     i, j, total: integer;
14 begin
15     total := 0;
16     for i := 1 to 2 do
17         begin
18             classes[i].id := i;
19             for j := 1 to 3 do
20                 begin
21                     classes[i].scores[j] := i * j;
22                     total := total + classes[i].scores[j];
23                 end;
24             end;
25
26             firstStudent := classes[1];
27             copiedStudent := firstStudent;
28             classes[2] := copiedStudent;
29         end.
30

```

```

test > milestone-3 > E tc6.txt_SEMANTIC.txt
323 === Decorated AST ===
324 Program(ArrayRecordValid), tab_index=41
325 |-- DeclarationPart
326 | |-- TypeDecl(ScoreIndex) : type=ScoreIndex, tab_index=42
327 | | \-- RangeType : type=ScoreIndex
328 | | | \-- Literal(1) value=1 literal=Integer : type=Integer
329 | | | \-- Literal(3) value=3 literal=Integer : type=Integer
330 | |-- TypeDecl(ScoreArray) : type=ScoreArray, tab_index=43
331 | | \-- ArrayType : type=ScoreArray
332 | | | \-- TypeRef(ScoreIndex) : type=ScoreIndex, tab_index=42
333 | | | \-- TypeRef(integer) : type=Integer, tab_index=32
334 | |-- TypeDecl(Student) : type=Student, tab_index=46
335 | | \-- RecordType : type=Student
336 | | | \-- FieldDecl(id) : type=Integer, tab_index=44
337 | | | | \-- TypeRef(integer) : type=Integer, tab_index=32
338 | | | \-- FieldDecl(scores) : type=ScoreArray, tab_index=45
339 | | | | \-- TypeRef(ScoreArray) : type=ScoreArray, tab_index=43
340 | |-- TypeDecl(Classroom) : type=Classroom, tab_index=47
341 | | \-- ArrayType : type=Classroom
342 | | | \-- RangeType : type=subrange(Integer)
343 | | | | \-- Literal(1) value=1 literal=Integer : type=Integer
344 | | | | \-- Literal(2) value=2 literal=Integer : type=Integer
345 | | | | \-- TypeRef(Student) : type=Student, tab_index=46
346 | |-- VarDecl(classes) : type=Classroom, tab_index=48
347 | | \-- TypeRef(Classroom) : type=Classroom, tab_index=47
348 | |-- VarDecl(firstStudent) : type=Student, tab_index=49
349 | | \-- TypeRef(Student) : type=Student, tab_index=46
350 | |-- VarDecl(copiedStudent) : type=Student, tab_index=50
351 | | \-- TypeRef(Student) : type=Student, tab_index=46
352 | |-- VarDecl(i) : type=Integer, tab_index=51
353 | | \-- TypeRef(integer) : type=Integer, tab_index=32
354 | |-- VarDecl(j) : type=Integer, tab_index=52
355 | | \-- TypeRef(integer) : type=Integer, tab_index=32
356 | \-- VarDecl(total) : type=Integer, tab_index=53
357 | | \-- TypeRef(integer) : type=Integer, tab_index=32
358 \-- CompoundStatement
359 | |-- AssignStatement : type=Integer, tab_index=53
360 | | \-- Variable(total) : type=Integer, tab_index=53
361 | | \-- Literal(0) value=0 literal=Integer : type=Integer
362 | |-- ForStatement(i) op=tosy, tab_index=51
363 | | \-- Literal(1) value=1 literal=Integer : type=Integer
364 | | \-- Literal(2) value=2 literal=Integer : type=Integer
365 | | \-- CompoundStatement
366 | | | |-- AssignStatement : type=Integer, tab_index=44
367 | | | | \-- Variable(classes) : type=Integer, tab_index=44
368 | | | | | \-- IndexComponent
369 | | | | | | \-- Variable(i) : type=Integer, tab_index=51
370 | | | | | | \-- FieldComponent(id)
371 | | | | | \-- Variable(i) : type=Integer, tab_index=51
372 | | | |-- ForStatement(j) op=tosy, tab_index=52
373 | | | | \-- Literal(1) value=1 literal=Integer : type=Integer
374 | | | | \-- Literal(3) value=3 literal=Integer : type=Integer
375 | | | | \-- CompoundStatement
376 | | | | |-- AssignStatement : type=Integer, tab_index=45
377 | | | | | \-- Variable(classes) : type=Integer, tab_index=45
378 | | | | | | \-- IndexComponent
379 | | | | | | | \-- Variable(i) : type=Integer, tab_index=51
380 | | | | | | | \-- FieldComponent(scores)
381 | | | | | | | \-- IndexComponent
382 | | | | | | | \-- Variable(j) : type=Integer, tab_index=52
383 | | | | | \-- BinaryOp op=times : type=Integer
384 | | | | | | \-- Variable(i) : type=Integer, tab_index=51
385 | | | | | | \-- Variable(j) : type=Integer, tab_index=52
386 | | | | |-- AssignStatement : type=Integer, tab_index=53
387 | | | | | \-- Variable(total) : type=Integer, tab_index=53
388 | | | | | \-- BinaryOp op=plus : type=Integer
389 | | | | | | \-- Variable(total) : type=Integer, tab_index=53
390 | | | | | | \-- Variable(classes) : type=Integer, tab_index=45
391 | | | | | | | \-- IndexComponent
392 | | | | | | | \-- Variable(i) : type=Integer, tab_index=51

```



```

test > milestone-3 > tc7.txt_SEMANTIC.txt
58   └─ period
59
60   === Decorated AST ===
61   Program(UndeclaredAndUninitializedError), tab_index=41
62   |-- DeclarationPart
63   |   |-- VarDecl(x) : type=Integer, tab_index=42
64   |   |   |-- TypeRef(integer) : type=Integer, tab_index=32
65   |   |-- VarDecl(y) : type=Integer, tab_index=43
66   |   |   |-- TypeRef(integer) : type=Integer, tab_index=32
67   |-- CompoundStatement
68   |   |-- AssignStatement : type=Integer, tab_index=42
69   |   |   |-- Variable(x) : type=Integer, tab_index=42
70   |   |   |-- BinaryOp op=plus
71   |   |   |   |-- Variable(missingValue)
72   |   |   |   |-- Literal(1) value=1 literal=Integer : type=Integer
73   |   |-- AssignStatement : type=Integer, tab_index=43
74   |   |   |-- Variable(y) : type=Integer, tab_index=43
75   |   |-- BinaryOp op=plus
76   |   |   |-- Variable(x) : type=Integer, tab_index=42
77   |   |   |-- Variable(notDeclared)
78   |-- EmptyStatement
79

```

```

=== Symbol Table: tab ===
idx identifier obj type ref nrm lev adr link init value
0 and reserved unknown 0 1 0 0 0 yes
1 array reserved unknown 0 1 0 0 0 yes
2 begin reserved unknown 0 1 0 0 1 yes
3 case reserved unknown 0 1 0 0 2 yes
4 const reserved unknown 0 1 0 0 3 yes
5 div reserved unknown 0 1 0 0 4 yes
6 downto reserved unknown 0 1 0 0 5 yes
7 do reserved unknown 0 1 0 0 6 yes
8 else reserved unknown 0 1 0 0 7 yes
9 end reserved unknown 0 1 0 0 8 yes
10 for reserved unknown 0 1 0 0 9 yes
11 function reserved unknown 0 1 0 0 10 yes
12 if reserved unknown 0 1 0 0 11 yes
13 mod reserved unknown 0 1 0 0 12 yes
14 not reserved unknown 0 1 0 0 13 yes
15 of reserved unknown 0 1 0 0 14 yes
16 or reserved unknown 0 1 0 0 15 yes
17 procedure reserved unknown 0 1 0 0 16 yes
18 program reserved unknown 0 1 0 0 17 yes
19 record reserved unknown 0 1 0 0 18 yes
20 repeat reserved unknown 0 1 0 0 19 yes
21 integer reserved unknown 0 1 0 0 20 yes
22 real reserved unknown 0 1 0 0 21 yes
23 boolean reserved unknown 0 1 0 0 22 yes
24 char reserved unknown 0 1 0 0 23 yes
25 string reserved unknown 0 1 0 0 24 yes
26 then reserved unknown 0 1 0 0 25 yes
27 to reserved unknown 0 1 0 0 26 yes
28 type reserved unknown 0 1 0 0 27 yes
29 until reserved unknown 0 1 0 0 28 yes
30 var reserved unknown 0 1 0 0 29 yes
31 while reserved unknown 0 1 0 0 30 yes
32 Integer type Integer 0 1 0 0 31 yes
33 Real type Real 0 1 0 0 32 yes
34 Char type Char 0 1 0 0 33 yes
35 Boolean type Boolean 0 1 0 0 34 yes
36 String type String 0 1 0 0 35 yes
37 True constant Boolean 0 1 0 1 36 yes true
38 False constant Boolean 0 1 0 0 37 yes false
39 readln procedure unknown 0 1 0 0 38 yes
40 writeln procedure unknown 0 1 0 0 39 yes
41 UndeclaredAndUninitializedError program unknown 0 1 0 0 40 yes
42 x variable Integer 0 1 0 0 41 no
43 y variable Integer 0 1 0 1 42 no

=== Symbol Table: btab ===
idx last lpar psze vsze
0 43 0 0 2

=== Symbol Table: atab ===
idx xtyp etyp eref low high elsz size

=== Semantic Errors ===
5:8: Error: Semantic error: Identifier 'missingValue' is not declared
5:8: Error: Semantic error: Operator plus requires numeric operands
6:8: Error: Semantic error: Variable 'x' may be used before initialization
6:12: Error: Semantic error: Identifier 'notDeclared' is not declared
6:8: Error: Semantic error: Operator plus requires numeric operands

```

Test Case 8: tes penolakan deklarasi ulang pada scope yang sama dan penolakan redeclaration terhadap predefined identifier

```
test > milestone-3 > tc8.txt
1  program RedeclarationError;
2  var
3    x: integer;
4    x: real;
5    Integer: integer;
6    true: boolean;
7  begin
8    x := 1;
9  end.
10
```

```

test > milestone-3 > tc8.txt_SEMANTIC.txt
47 | | | semicolon
48 | | | <statement>
49 | | endsy
50 | period
51
52 === Decorated AST ===
53 Program(RedeclarationError), tab_index=41
54 |-- DeclarationPart
55 | |-- VarDecl(x) : type=Integer, tab_index=42
56 | | \-- TypeRef(integer) : type=Integer, tab_index=32
57 | |-- VarDecl(x) : type=Real, tab_index=42
58 | | \-- TypeRef(real) : type=Real, tab_index=33
59 | |-- VarDecl(Integer) : type=Integer, tab_index=32
60 | | \-- TypeRef(integer) : type=Integer, tab_index=32
61 | \-- VarDecl(true) : type=Boolean, tab_index=37
62 | \-- TypeRef(boolean) : type=Boolean, tab_index=35
63 \-- CompoundStatement
64 | |-- AssignStatement : type=Integer, tab_index=42
65 | | |-- Variable(x) : type=Integer, tab_index=42
66 | | \-- Literal(1) value=1 literal=Integer : type=Integer
67 | \-- EmptyStatement
68
69 === Symbol Table: tab ===
70 idx identifier obj type ref nrm lev adr link init value
71 0 and reserved unknown 0 1 0 0 0 yes
72 1 array reserved unknown 0 1 0 0 0 yes
73 2 begin reserved unknown 0 1 0 0 1 yes
74 3 case reserved unknown 0 1 0 0 2 yes
75 4 const reserved unknown 0 1 0 0 3 yes
76 5 div reserved unknown 0 1 0 0 4 yes
77 6 downto reserved unknown 0 1 0 0 5 yes
78 7 do reserved unknown 0 1 0 0 6 yes
79 8 else reserved unknown 0 1 0 0 7 yes
80 9 end reserved unknown 0 1 0 0 8 yes
81 10 for reserved unknown 0 1 0 0 9 yes
82 11 function reserved unknown 0 1 0 0 10 yes
83 12 if reserved unknown 0 1 0 0 11 yes
84 13 mod reserved unknown 0 1 0 0 12 yes
85 14 not reserved unknown 0 1 0 0 13 yes
86 15 of reserved unknown 0 1 0 0 14 yes
87 16 or reserved unknown 0 1 0 0 15 yes
88 17 procedure reserved unknown 0 1 0 0 16 yes
89 18 program reserved unknown 0 1 0 0 17 yes
90 19 record reserved unknown 0 1 0 0 18 yes
91 20 repeat reserved unknown 0 1 0 0 19 yes
92 21 integer reserved unknown 0 1 0 0 20 yes
93 22 real reserved unknown 0 1 0 0 21 yes
94 23 boolean reserved unknown 0 1 0 0 22 yes
95 24 char reserved unknown 0 1 0 0 23 yes
96 25 string reserved unknown 0 1 0 0 24 yes
97 26 then reserved unknown 0 1 0 0 25 yes
98 27 to reserved unknown 0 1 0 0 26 yes
99 28 type reserved unknown 0 1 0 0 27 yes
100 29 until reserved unknown 0 1 0 0 28 yes
101 30 var reserved unknown 0 1 0 0 29 yes
102 31 while reserved unknown 0 1 0 0 30 yes
103 32 Integer type Integer 0 1 0 0 31 yes
104 33 Real type Real 0 1 0 0 32 yes
105 34 Char type Char 0 1 0 0 33 yes
106 35 Boolean type Boolean 0 1 0 0 34 yes
107 36 String type String 0 1 0 0 35 yes
108 37 True constant Boolean 0 1 0 1 36 yes true
109 38 False constant Boolean 0 1 0 0 37 yes false
110 39 readln procedure unknown 0 1 0 0 38 yes
111 40 writeln procedure unknown 0 1 0 0 39 yes
112 41 RedeclarationError program unknown 0 1 0 0 40 yes
113 42 x variable Integer 0 1 0 0 41 yes
114

```

```

=== Symbol Table: btab ===
idx last  lpar  psze  vsze
0  42  0  0  1

=== Symbol Table: atab ===
idx xtyp  etyp  eref  low high  elsz  size

=== Semantic Errors ===
4:3: Error: Semantic error: Identifier 'x' is already declared in the current scope
5:3: Error: Semantic error: Identifier 'Integer' is already declared in the current scope
6:3: Error: Semantic error: Identifier 'true' is already declared in the current scope

```

Test Case 9: tes penolakan assignment yang tidak kompatibel dan operasi dengan operand yang tidak sesuai tipe

```

test > milestone-3 > tc9.txt
1  program AssignmentAndOperatorTypeError;
2  var
3      i, j: integer;
4      b: boolean;
5      c: char;
6      s: string;
7  begin
8      j := 1;
9      i := 'x';
10     b := 10;
11     c := true;
12     s := 'a' + j;
13     i := b * j;
14 end.

```

```

test > milestone-3 > tc9.txt_SEMANTIC.txt
118 |   |   \Statement
119 |   |   |   endsy
120 |   |   |   period
121 |
122 |   === Decorated AST ===
123 |   Program(AssignmentAndOperatorTypeError), tab_index=41
124 |   |-- DeclarationPart
125 |   |   |-- VarDecl(i) : type=Integer, tab_index=42
126 |   |   |   \-- TypeRef(integer) : type=Integer, tab_index=32
127 |   |   |-- VarDecl(j) : type=Integer, tab_index=43
128 |   |   |   \-- TypeRef(integer) : type=Integer, tab_index=32
129 |   |   |-- VarDecl(b) : type=Boolean, tab_index=44
130 |   |   |   \-- TypeRef(boolean) : type=Boolean, tab_index=35
131 |   |   |-- VarDecl(c) : type=Char, tab_index=45
132 |   |   |   \-- TypeRef(char) : type=Char, tab_index=34
133 |   |   \-- VarDecl(s) : type=String, tab_index=46
134 |   |       \-- TypeRef(string) : type=String, tab_index=36
135 |   \-- CompoundStatement
136 |       |-- AssignStatement : type=Integer, tab_index=43
137 |       |   |-- Variable(j) : type=Integer, tab_index=43
138 |       |   |   \-- Literal(1) value=1 literal=Integer : type=Integer
139 |       |   |-- AssignStatement : type=Integer, tab_index=42
140 |       |   |   |-- Variable(i) : type=Integer, tab_index=42
141 |       |   |   |   \-- Literal('x') value='x' literal=Char : type=Char
142 |       |   |   |-- AssignStatement : type=Boolean, tab_index=44
143 |       |   |   |   |-- Variable(b) : type=Boolean, tab_index=44
144 |       |   |   |   |   \-- Literal(10) value=10 literal=Integer : type=Integer
145 |       |   |   |-- AssignStatement : type=Char, tab_index=45
146 |       |   |   |   |-- Variable(c) : type=Char, tab_index=45
147 |       |   |   |   |   \-- Variable(true) : type=Boolean, tab_index=37
148 |       |   |   |-- AssignStatement : type=String, tab_index=46
149 |       |   |   |   |-- Variable(s) : type=String, tab_index=46
150 |       |   |   |   |   \-- BinaryOp op=plus
151 |       |   |   |   |   |   |-- Literal('a') value='a' literal=Char : type=Char
152 |       |   |   |   |   |   |   \-- Variable(j) : type=Integer, tab_index=43
153 |       |   |   |-- AssignStatement : type=Integer, tab_index=42
154 |       |   |   |   |-- Variable(i) : type=Integer, tab_index=42
155 |       |   |   |   |   \-- BinaryOp op=times
156 |       |   |   |   |   |   |-- Variable(b) : type=Boolean, tab_index=44
157 |       |   |   |   |   |   |   \-- Variable(j) : type=Integer, tab_index=43
158 |       |   \-- EmptyStatement
159 |

```

```

test > milestone-3 > E tc9.txt_SEMANTIC.txt
135 \-- CompoundStatement
137 | \-- variable(j) : type=integer; tab_index=45
158 \-- EmptyStatement
159
160 === Symbol Table: tab ===
161 idx identifier obj type ref nrm lev adr link init value
162 0 and reserved unknown 0 1 0 0 0 yes
163 1 array reserved unknown 0 1 0 0 0 yes
164 2 begin reserved unknown 0 1 0 0 1 yes
165 3 case reserved unknown 0 1 0 0 2 yes
166 4 const reserved unknown 0 1 0 0 3 yes
167 5 div reserved unknown 0 1 0 0 4 yes
168 6 downto reserved unknown 0 1 0 0 5 yes
169 7 do reserved unknown 0 1 0 0 6 yes
170 8 else reserved unknown 0 1 0 0 7 yes
171 9 end reserved unknown 0 1 0 0 8 yes
172 10 for reserved unknown 0 1 0 0 9 yes
173 11 function reserved unknown 0 1 0 0 10 yes
174 12 if reserved unknown 0 1 0 0 11 yes
175 13 mod reserved unknown 0 1 0 0 12 yes
176 14 not reserved unknown 0 1 0 0 13 yes
177 15 of reserved unknown 0 1 0 0 14 yes
178 16 or reserved unknown 0 1 0 0 15 yes
179 17 procedure reserved unknown 0 1 0 0 16 yes
180 18 program reserved unknown 0 1 0 0 17 yes
181 19 record reserved unknown 0 1 0 0 18 yes
182 20 repeat reserved unknown 0 1 0 0 19 yes
183 21 integer reserved unknown 0 1 0 0 20 yes
184 22 real reserved unknown 0 1 0 0 21 yes
185 23 boolean reserved unknown 0 1 0 0 22 yes
186 24 char reserved unknown 0 1 0 0 23 yes
187 25 string reserved unknown 0 1 0 0 24 yes
188 26 then reserved unknown 0 1 0 0 25 yes
189 27 to reserved unknown 0 1 0 0 26 yes
190 28 type reserved unknown 0 1 0 0 27 yes
191 29 until reserved unknown 0 1 0 0 28 yes
192 30 var reserved unknown 0 1 0 0 29 yes
193 31 while reserved unknown 0 1 0 0 30 yes
194 32 Integer type Integer 0 1 0 0 31 yes
195 33 Real type Real 0 1 0 0 32 yes
196 34 Char type Char 0 1 0 0 33 yes
197 35 Boolean type Boolean 0 1 0 0 34 yes
198 36 String type String 0 1 0 0 35 yes
199 37 True constant Boolean 0 1 0 1 36 yes true
200 38 False constant Boolean 0 1 0 0 37 yes false
201 39 readln procedure unknown 0 1 0 0 38 yes
202 40 writeln procedure unknown 0 1 0 0 39 yes
203 41 AssignmentAndOperatorTypeError program unknown 0 1 0 0 40 yes
204 42 i variable Integer 0 1 0 0 41 no
205 43 j variable Integer 0 1 0 1 42 yes
206 44 b variable Boolean 0 1 0 2 43 no
207 45 c variable Char 0 1 0 3 44 no
208 46 s variable String 0 1 0 4 45 no
209
210 === Symbol Table: btab ===
211 idx last lpar psze vsze
212 0 46 0 0 5
213
214 === Symbol Table: atab ===
215 idx xtyp etyp eref low high elsz size
216
217 === Semantic Errors ===
218 9:3: Error: Semantic error: Cannot assign Char to Integer
219 10:3: Error: Semantic error: Cannot assign Integer to Boolean
220 11:3: Error: Semantic error: Cannot assign Boolean to Char
221 12:8: Error: Semantic error: Operator plus requires numeric operands
222 13:8: Error: Semantic error: Variable 'b' may be used before initialization
223 13:8: Error: Semantic error: Operator times requires numeric operands
224

```


Test Case 10: tes penolakan akses array dengan indeks bertipe salah atau konstanta di luar rentang, serta akses field record yang tidak ada atau akses field pada non-record

```
test > milestone-3 > tc10.txt
1  program ArrayAndRecordAccessError;
2  type
3      IndexRange == 1..3;
4      Values == array[IndexRange] of integer;
5      Point == record
6          | x, y: integer;
7      end;
8  var
9      values: Values;
10     p: Point;
11     i: integer;
12     r: real;
13 begin
14     r := 2.5;
15     values[r] := 10;
16     values[4] := 20;
17     p.z := 10;
18     i.x := 1;
19 end.
20
```

```

test > milestone-3 > tc10.txt_SEMANTIC.txt
160 |   └─ endsy
161 |   └─ period
162
163 === Decorated AST ===
164 Program(ArrayAndRecordAccessError), tab_index=41
165 |-- DeclarationPart
166 |   |-- TypeDecl(IndexRange) : type=IndexRange, tab_index=42
167 |   |   \-- RangeType : type=IndexRange
168 |   |       |-- Literal(1) value=1 literal=Integer : type=Integer
169 |   |       \-- Literal(3) value=3 literal=Integer : type=Integer
170 |   |-- TypeDecl(Values) : type=Values, tab_index=43
171 |   |   \-- ArrayType : type=Values
172 |   |       |-- TypeRef(IndexRange) : type=IndexRange, tab_index=42
173 |   |       \-- TypeRef(integer) : type=Integer, tab_index=32
174 |   |-- TypeDecl(Point) : type=Point, tab_index=46
175 |   |   \-- RecordType : type=Point
176 |   |       |-- FieldDecl(x) : type=Integer, tab_index=44
177 |   |       |   \-- TypeRef(integer) : type=Integer, tab_index=32
178 |   |       \-- FieldDecl(y) : type=Integer, tab_index=45
179 |   |           \-- TypeRef(integer) : type=Integer, tab_index=32
180 |   |-- VarDecl(values) : type=Values, tab_index=47
181 |   |   \-- TypeRef(Values) : type=Values, tab_index=43
182 |   |-- VarDecl(p) : type=Point, tab_index=48
183 |   |   \-- TypeRef(Point) : type=Point, tab_index=46
184 |   |-- VarDecl(i) : type=Integer, tab_index=49
185 |   |   \-- TypeRef(integer) : type=Integer, tab_index=32
186 |   \-- VarDecl(r) : type=Real, tab_index=50
187 |       \-- TypeRef(real) : type=Real, tab_index=33
188 |-- CompoundStatement
189 |   |-- AssignStatement : type=Real, tab_index=50
190 |   |   |-- Variable(r) : type=Real, tab_index=50
191 |   |   \-- Literal(2.5) value=2.5 literal=Real : type=Real
192 |   |-- AssignStatement : type=Integer, tab_index=47
193 |   |   |-- Variable(values) : type=Integer, tab_index=47
194 |   |   |   \-- IndexComponent
195 |   |   |       \-- Variable(r) : type=Real, tab_index=50
196 |   |   \-- Literal(10) value=10 literal=Integer : type=Integer
197 |   |-- AssignStatement : type=Integer, tab_index=47
198 |   |   |-- Variable(values) : type=Integer, tab_index=47
199 |   |   |   \-- IndexComponent
200 |   |   |       \-- Literal(4) value=4 literal=Integer : type=Integer
201 |   |   \-- Literal(20) value=20 literal=Integer : type=Integer
202 |   |-- AssignStatement, tab_index=48
203 |   |   |-- Variable(p), tab_index=48
204 |   |   |   \-- FieldComponent(z)
205 |   |   \-- Literal(10) value=10 literal=Integer : type=Integer
206 |   |-- AssignStatement, tab_index=49
207 |   |   |-- Variable(i), tab_index=49
208 |   |   |   \-- FieldComponent(x)
209 |   |   \-- Literal(1) value=1 literal=Integer : type=Integer
210 |   \-- EmptyStatement
211

```

```

212 === Symbol Table: tab ===
213 idx identifier obj type ref nrm lev adr link init value
214 0 and reserved unknown 0 1 0 0 0 yes
215 1 array reserved unknown 0 1 0 0 0 yes
216 2 begin reserved unknown 0 1 0 0 1 yes
217 3 case reserved unknown 0 1 0 0 2 yes
218 4 const reserved unknown 0 1 0 0 3 yes
219 5 div reserved unknown 0 1 0 0 4 yes
220 6 downto reserved unknown 0 1 0 0 5 yes
221 7 do reserved unknown 0 1 0 0 6 yes
222 8 else reserved unknown 0 1 0 0 7 yes
223 9 end reserved unknown 0 1 0 0 8 yes
224 10 for reserved unknown 0 1 0 0 9 yes
225 11 function reserved unknown 0 1 0 0 10 yes
226 12 if reserved unknown 0 1 0 0 11 yes
227 13 mod reserved unknown 0 1 0 0 12 yes
228 14 not reserved unknown 0 1 0 0 13 yes
229 15 of reserved unknown 0 1 0 0 14 yes
230 16 or reserved unknown 0 1 0 0 15 yes
231 17 procedure reserved unknown 0 1 0 0 16 yes
232 18 program reserved unknown 0 1 0 0 17 yes
233 19 record reserved unknown 0 1 0 0 18 yes
234 20 repeat reserved unknown 0 1 0 0 19 yes
235 21 integer reserved unknown 0 1 0 0 20 yes
236 22 real reserved unknown 0 1 0 0 21 yes
237 23 boolean reserved unknown 0 1 0 0 22 yes
238 24 char reserved unknown 0 1 0 0 23 yes
239 25 string reserved unknown 0 1 0 0 24 yes
240 26 then reserved unknown 0 1 0 0 25 yes
241 27 to reserved unknown 0 1 0 0 26 yes
242 28 type reserved unknown 0 1 0 0 27 yes
243 29 until reserved unknown 0 1 0 0 28 yes
244 30 var reserved unknown 0 1 0 0 29 yes
245 31 while reserved unknown 0 1 0 0 30 yes
246 32 Integer type Integer 0 1 0 0 31 yes
247 33 Real type Real 0 1 0 0 32 yes
248 34 Char type Char 0 1 0 0 33 yes
249 35 Boolean type Boolean 0 1 0 0 34 yes
250 36 String type String 0 1 0 0 35 yes
251 37 True constant Boolean 0 1 0 1 36 yes true
252 38 False constant Boolean 0 1 0 0 37 yes false
253 39 readln procedure unknown 0 1 0 0 38 yes
254 40 writeln procedure unknown 0 1 0 0 39 yes
255 41 ArrayAndRecordAccessError program unknown 0 1 0 0 40 yes
256 42 IndexRange type IndexRange 0 1 0 0 41 yes
257 43 Values type Values 0 1 0 0 42 yes
258 44 x field Integer 0 1 1 0 0 yes
259 45 y field Integer 0 1 1 1 44 yes
260 46 Point type Point 1 1 0 0 43 yes
261 47 values variable Values 0 1 0 0 46 yes
262 48 p variable Point 1 1 0 3 47 yes
263 49 i variable Integer 0 1 0 5 48 yes
264 50 r variable Real 0 1 0 6 49 yes
265

```

```

266  === Symbol Table: btab ===
267  idx last  lpar  psze  vsze
268  0  50  0  0  8
269  1  45  0  0  2
270
271  === Symbol Table: atab ===
272  idx xtyp  etyp  eref  low high  elsz  size
273  0  IndexRange Integer 0  1  3  1  3
274
275  === Semantic Errors ===
276  15:10: Error: Semantic error: Array index type Real is incompatible with IndexRange
277  16:10: Error: Semantic error: Array index value is outside bounds 1..3
278  17:4: Error: Semantic error: Record field 'z' is not declared
279  18:4: Error: Semantic error: Field access requires a Record type
280

```

Test Case 11: tes penolakan kondisi control flow yang bukan Boolean, variabel for bertipe Real, assignment ke konstanta, dan pemanggilan prosedur/fungsi dengan argumen yang salah

```

test > milestone-3 > tc11.txt
1  program ControlCallAndConstError;
2  const
3      Limit == 10;
4  var
5      i, x: integer;
6      r: real;
7
8      procedure SetX(value: integer);
9      begin
10         x := value;
11     end;
12
13     function Twice(value: integer): integer;
14     begin
15         Twice := value * 2;
16     end;
17
18     begin
19         i := 0;
20         x := 0;
21         r := 1.5;
22
23         Limit := 11;
24
25         if i then
26         begin
27             i := i + 1;
28         end;
29
30         while r do
31         begin
32             r := r - 1.0;
33         end;
34
35         for r := 1.0 to 3.0 do
36         begin
37             i := i + 1;
38         end;
39
40         SetX(true);
41         x := Twice('a');
42         Twice(2);
43     end.
44

```

```

test > milestone-3 > tc11.txt_SEMANTIC.txt
304   └─ period
305
306   === Decorated AST ===
307   Program(ControlCallAndConstError), tab_index=41
308   |-- DeclarationPart
309   |   |-- ConstDecl(Limit) : type=Integer, tab_index=42
310   |   |   |-- Literal(10) value=10 literal=Integer : type=Integer
311   |   |-- VarDecl(i) : type=Integer, tab_index=43
312   |   |   |-- TypeRef(integer) : type=Integer, tab_index=32
313   |   |-- VarDecl(x) : type=Integer, tab_index=44
314   |   |   |-- TypeRef(integer) : type=Integer, tab_index=32
315   |   |-- VarDecl(r) : type=Real, tab_index=45
316   |   |   |-- TypeRef(real) : type=Real, tab_index=33
317   |   |-- ProcedureDecl(SetX), tab_index=46, block=1
318   |   |   |-- ParameterGroup
319   |   |   |   |-- VarDecl(value) : type=Integer, tab_index=47, lev=1
320   |   |   |   |   |-- TypeRef(integer) : type=Integer, tab_index=32
321   |   |   |-- DeclarationPart
322   |   |   |-- CompoundStatement, lev=1, block=1
323   |   |   |   |-- AssignStatement : type=Integer, tab_index=44
324   |   |   |   |   |-- Variable(x) : type=Integer, tab_index=44, lev=1
325   |   |   |   |   |-- Variable(value) : type=Integer, tab_index=47, lev=1
326   |   |   |   |-- EmptyStatement
327   |   |-- FunctionDecl(Twice) : type=Integer, tab_index=48, block=2
328   |   |   |-- TypeRef(integer) : type=Integer, tab_index=32
329   |   |   |-- ParameterGroup
330   |   |   |   |-- VarDecl(value) : type=Integer, tab_index=49, lev=1
331   |   |   |   |   |-- TypeRef(integer) : type=Integer, tab_index=32
332   |   |   |-- DeclarationPart
333   |   |   |-- CompoundStatement, lev=1, block=2
334   |   |   |   |-- AssignStatement : type=Integer, tab_index=48
335   |   |   |   |   |-- Variable(Twice) : type=Integer, tab_index=48, lev=1
336   |   |   |   |   |-- BinaryOp op=times : type=Integer
337   |   |   |   |   |-- Variable(value) : type=Integer, tab_index=49, lev=1
338   |   |   |   |   |-- Literal(2) value=2 literal=Integer : type=Integer
339   |   |   |   |-- EmptyStatement
340   |-- CompoundStatement
341   |   |-- AssignStatement : type=Integer, tab_index=43
342   |   |   |-- Variable(i) : type=Integer, tab_index=43
343   |   |   |-- Literal(0) value=0 literal=Integer : type=Integer
344   |   |-- AssignStatement : type=Integer, tab_index=44
345   |   |   |-- Variable(x) : type=Integer, tab_index=44
346   |   |   |-- Literal(0) value=0 literal=Integer : type=Integer
347   |   |-- AssignStatement : type=Real, tab_index=45
348   |   |   |-- Variable(r) : type=Real, tab_index=45
349   |   |   |-- Literal(1.5) value=1.5 literal=Real : type=Real
350   |   |-- AssignStatement : type=Integer, tab_index=42
351   |   |   |-- Variable(Limit) : type=Integer, tab_index=42
352   |   |   |-- Literal(11) value=11 literal=Integer : type=Integer
353   |   |-- IfStatement : type=Integer
354   |   |   |-- Variable(i) : type=Integer, tab_index=43
355   |   |   |-- CompoundStatement
356   |   |   |   |-- AssignStatement : type=Integer, tab_index=43
357   |   |   |   |   |-- Variable(i) : type=Integer, tab_index=43
358   |   |   |   |   |-- BinaryOp op=plus : type=Integer
359   |   |   |   |   |-- Variable(i) : type=Integer, tab_index=43
360   |   |   |   |   |-- Literal(1) value=1 literal=Integer : type=Integer
361   |   |   |   |-- EmptyStatement
362   |   |-- WhileStatement : type=Real

```

```

362 |-- WhileStatement : type=Real
363 | |-- Variable(r) : type=Real, tab_index=45
364 | |-- CompoundStatement
365 | | |-- AssignStatement : type=Real, tab_index=45
366 | | |-- Variable(r) : type=Real, tab_index=45
367 | | |-- BinaryOp op=minus : type=Real
368 | | |-- Variable(r) : type=Real, tab_index=45
369 | | |-- Literal(1.0) value=1.0 literal=Real : type=Real
370 | |-- EmptyStatement
371 |-- ForStatement(r) op=tosy, tab_index=45
372 | |-- Literal(1.0) value=1.0 literal=Real : type=Real
373 | |-- Literal(3.0) value=3.0 literal=Real : type=Real
374 | |-- CompoundStatement
375 | | |-- AssignStatement : type=Integer, tab_index=43
376 | | |-- Variable(i) : type=Integer, tab_index=43
377 | | |-- BinaryOp op=plus : type=Integer
378 | | |-- Variable(i) : type=Integer, tab_index=43
379 | | |-- Literal(1) value=1 literal=Integer : type=Integer
380 | |-- EmptyStatement
381 |-- Call(SetX), tab_index=46
382 | |-- Variable(true) : type=Boolean, tab_index=37
383 |-- AssignStatement : type=Integer, tab_index=44
384 | |-- Variable(x) : type=Integer, tab_index=44
385 | |-- Call(Twice) : type=Integer, tab_index=48
386 | |-- Literal('a') value='a' literal=Char : type=Char
387 |-- Call(Twice) : type=Integer, tab_index=48
388 | |-- Literal(2) value=2 literal=Integer : type=Integer
389 |-- EmptyStatement
390

```

```

391  === Symbol Table: tab ===
392  idx identifier obj type    ref nrm lev adr link  init  value
393  0   and reserved   unknown 0   1   0   0   0   yes
394  1   array reserved   unknown 0   1   0   0   0   yes
395  2   begin reserved   unknown 0   1   0   0   1   yes
396  3   case reserved   unknown 0   1   0   0   2   yes
397  4   const reserved   unknown 0   1   0   0   3   yes
398  5   div reserved   unknown 0   1   0   0   4   yes
399  6   downto reserved   unknown 0   1   0   0   5   yes
400  7   do reserved   unknown 0   1   0   0   6   yes
401  8   else reserved   unknown 0   1   0   0   7   yes
402  9   end reserved   unknown 0   1   0   0   8   yes
403  10  for reserved   unknown 0   1   0   0   9   yes
404  11  function reserved   unknown 0   1   0   0  10   yes
405  12  if reserved   unknown 0   1   0   0  11   yes
406  13  mod reserved   unknown 0   1   0   0  12   yes
407  14  not reserved   unknown 0   1   0   0  13   yes
408  15  of reserved   unknown 0   1   0   0  14   yes
409  16  or reserved   unknown 0   1   0   0  15   yes
410  17  procedure reserved   unknown 0   1   0   0  16   yes
411  18  program reserved   unknown 0   1   0   0  17   yes
412  19  record reserved   unknown 0   1   0   0  18   yes
413  20  repeat reserved   unknown 0   1   0   0  19   yes
414  21  integer reserved   unknown 0   1   0   0  20   yes
415  22  real reserved   unknown 0   1   0   0  21   yes
416  23  boolean reserved   unknown 0   1   0   0  22   yes
417  24  char reserved   unknown 0   1   0   0  23   yes
418  25  string reserved   unknown 0   1   0   0  24   yes
419  26  then reserved   unknown 0   1   0   0  25   yes
420  27  to reserved   unknown 0   1   0   0  26   yes
421  28  type reserved   unknown 0   1   0   0  27   yes
422  29  until reserved   unknown 0   1   0   0  28   yes
423  30  var reserved   unknown 0   1   0   0  29   yes
424  31  while reserved   unknown 0   1   0   0  30   yes
425  32  Integer type   Integer 0   1   0   0  31   yes
426  33  Real type     Real    0   1   0   0  32   yes
427  34  Char type     Char    0   1   0   0  33   yes
428  35  Boolean type   Boolean 0   1   0   0  34   yes
429  36  String type    String 0   1   0   0  35   yes
430  37  True constant Boolean 0   1   0   1  36   yes true
431  38  False constant Boolean 0   1   0   0  37   yes false
432  39  readln procedure unknown 0   1   0   0  38   yes
433  40  writeln procedure unknown 0   1   0   0  39   yes
434  41  ControlCallAndConstError program unknown 0   1   0   0  40   yes
435  42  Limit constant Integer 0   1   0   0  41   yes 10
436  43  i variable    Integer 0   1   0   0  42   yes
437  44  x variable    Integer 0   1   0   1  43   yes
438  45  r variable    Real    0   1   0   2  44   yes
439  46  SetX procedure unknown 1   1   0   4  45   yes
440  47  value parameter Integer 0   1   1   0   0   yes
441  48  Twice function Integer 2   1   0   4  46   yes
442  49  value parameter Integer 0   1   1   0   0   yes
443

```



```

444 === Symbol Table: btab ===
445   idx last  lpar  psze  vsze
446   0  48  0   0   4
447   1  47  47  1   0
448   2  49  49  1   0
449
450 === Symbol Table: atab ===
451   idx xtyp  etyp  eref  low high  elsz  size
452
453 === Semantic Errors ===
454 23:3: Error: Semantic error: Identifier 'Limit' is not an assignable target (constant)
455 25:3: Error: Semantic error: if condition must be Boolean
456 30:3: Error: Semantic error: while condition must be Boolean
457 35:3: Error: Semantic error: for iterator 'r' must be a simple ordinal non-Real type
458 40:8: Error: Semantic error: Argument 1 of 'SetX' expects Integer, got Boolean
459 41:14: Error: Semantic error: Argument 1 of 'Twice' expects Integer, got Char
460

```

Test Case 12: tes validasi deklarasi tipe, seperti batas subrange terbalik, subrange bertipe Real, array dengan index type invalid, dan enumerated type yang memakai identifier yang belum dideklarasikan

```

test > milestone-3 > tc12.txt
1  program TypeDeclarationAndEnumError;
2  type
3      Color == (Red, Green, Blue);
4      BadRangeOrder == 10..1;
5      BadRealRange == 1.5..3.5;
6      BadArray == array[BadRealRange] of integer;
7  var
8      x: BadRangeOrder;
9      color: Color;
10 begin
11     x := 5;
12     color := Red;
13 end.
14

```

```

test > milestone-3 > tc12.txt_SEMANTIC.txt
100  === Decorated AST ===
101  Program(TypeDeclarationAndEnumError), tab_index=41
102  |-- DeclarationPart
103  |   |-- TypeDecl(Color) : type=Color, tab_index=42
104  |   |   |-- EnumType : type=Color
105  |   |   |   |-- Literal(Red) value=Red literal=IdentifierConstant
106  |   |   |   |-- Literal(Green) value=Green literal=IdentifierConstant
107  |   |   |   |-- Literal(Blue) value=Blue literal=IdentifierConstant
108  |   |-- TypeDecl(BadRangeOrder) : type=BadRangeOrder, tab_index=43
109  |   |   |-- RangeType : type=BadRangeOrder
110  |   |   |   |-- Literal(10) value=10 literal=Integer : type=Integer
111  |   |   |   |-- Literal(1) value=1 literal=Integer : type=Integer
112  |   |-- TypeDecl(BadRealRange) : type=BadRealRange, tab_index=44
113  |   |   |-- RangeType : type=BadRealRange
114  |   |   |   |-- Literal(1.5) value=1.5 literal=Real : type=Real
115  |   |   |   |-- Literal(3.5) value=3.5 literal=Real : type=Real
116  |   |-- TypeDecl(BadArray) : type=BadArray, tab_index=45
117  |   |   |-- ArrayType : type=BadArray
118  |   |   |   |-- TypeRef(BadRealRange) : type=BadRealRange, tab_index=44
119  |   |   |   |-- TypeRef(integer) : type=Integer, tab_index=32
120  |   |-- VarDecl(x) : type=BadRangeOrder, tab_index=46
121  |   |   |-- TypeRef(BadRangeOrder) : type=BadRangeOrder, tab_index=43
122  |   |-- VarDecl(color) : type=Color, tab_index=47
123  |   |   |-- TypeRef(Color) : type=Color, tab_index=42
124  |-- CompoundStatement
125  |   |-- AssignStatement : type=BadRangeOrder, tab_index=46
126  |   |   |-- Variable(x) : type=BadRangeOrder, tab_index=46
127  |   |   |   |-- Literal(5) value=5 literal=Integer : type=Integer
128  |   |-- AssignStatement : type=Color, tab_index=47
129  |   |   |-- Variable(color) : type=Color, tab_index=47
130  |   |   |-- Variable(Red)
131  |   |-- EmptyStatement
132

```

```

133 === Symbol Table: tab ===
134 idx identifier obj type ref nrm lev adr link init value
135 0 and reserved unknown 0 1 0 0 0 yes
136 1 array reserved unknown 0 1 0 0 0 yes
137 2 begin reserved unknown 0 1 0 0 1 yes
138 3 case reserved unknown 0 1 0 0 2 yes
139 4 const reserved unknown 0 1 0 0 3 yes
140 5 div reserved unknown 0 1 0 0 4 yes
141 6 downto reserved unknown 0 1 0 0 5 yes
142 7 do reserved unknown 0 1 0 0 6 yes
143 8 else reserved unknown 0 1 0 0 7 yes
144 9 end reserved unknown 0 1 0 0 8 yes
145 10 for reserved unknown 0 1 0 0 9 yes
146 11 function reserved unknown 0 1 0 0 10 yes
147 12 if reserved unknown 0 1 0 0 11 yes
148 13 mod reserved unknown 0 1 0 0 12 yes
149 14 not reserved unknown 0 1 0 0 13 yes
150 15 of reserved unknown 0 1 0 0 14 yes
151 16 or reserved unknown 0 1 0 0 15 yes
152 17 procedure reserved unknown 0 1 0 0 16 yes
153 18 program reserved unknown 0 1 0 0 17 yes
154 19 record reserved unknown 0 1 0 0 18 yes
155 20 repeat reserved unknown 0 1 0 0 19 yes
156 21 integer reserved unknown 0 1 0 0 20 yes
157 22 real reserved unknown 0 1 0 0 21 yes
158 23 boolean reserved unknown 0 1 0 0 22 yes
159 24 char reserved unknown 0 1 0 0 23 yes
160 25 string reserved unknown 0 1 0 0 24 yes
161 26 then reserved unknown 0 1 0 0 25 yes
162 27 to reserved unknown 0 1 0 0 26 yes
163 28 type reserved unknown 0 1 0 0 27 yes
164 29 until reserved unknown 0 1 0 0 28 yes
165 30 var reserved unknown 0 1 0 0 29 yes
166 31 while reserved unknown 0 1 0 0 30 yes
167 32 Integer type Integer 0 1 0 0 31 yes
168 33 Real type Real 0 1 0 0 32 yes
169 34 Char type Char 0 1 0 0 33 yes
170 35 Boolean type Boolean 0 1 0 0 34 yes
171 36 String type String 0 1 0 0 35 yes
172 37 True constant Boolean 0 1 0 1 36 yes true
173 38 False constant Boolean 0 1 0 0 37 yes false
174 39 readln procedure unknown 0 1 0 0 38 yes
175 40 writeln procedure unknown 0 1 0 0 39 yes
176 41 TypeDeclarationAndEnumError program unknown 0 1 0 0 40 yes
177 42 Color type Color 0 1 0 0 41 yes
178 43 BadRangeOrder type BadRangeOrder 0 1 0 0 42 yes
179 44 BadRealRange type BadRealRange 0 1 0 0 43 yes
180 45 BadArray type BadArray 0 1 0 0 44 yes
181 46 x variable BadRangeOrder 0 1 0 0 45 no
182 47 color variable Color 0 1 0 1 46 no
183

```

```

184  === Symbol Table: btab ===
185  idx last  lpar  psze  vsze
186  0  47  0  0  2
187
188  === Symbol Table: atab ===
189  idx xtyp  etyp  eref  low high  elsz  size
190  0  BadRealRange  Integer 0  0  0  1  1
191
192  === Semantic Errors ===
193  3:13: Error: Semantic error: Enumerated identifier 'Red' must be declared before the enumerated type
194  3:18: Error: Semantic error: Enumerated identifier 'Green' must be declared before the enumerated type
195  3:25: Error: Semantic error: Enumerated identifier 'Blue' must be declared before the enumerated type
196  4:20: Error: Semantic error: Range lower bound is greater than upper bound
197  5:19: Error: Semantic error: Subrange cannot use Real bounds
198  6:15: Error: Semantic error: Array index type must be a simple non-Real type
199  11:3: Error: Semantic error: Cannot assign Integer to BadRangeOrder
200  12:12: Error: Semantic error: Identifier 'Red' is not declared
201

```

IV. Kesimpulan

Milestone 3 Arion Compiler telah mengimplementasikan semantic analysis sesuai inti spesifikasi. Program mampu mengubah parse tree menjadi AST, mendekorasi AST dengan atribut semantik, membentuk symbol table tab, btab, dan atab, menginisialisasi predefined identifiers, menerapkan aturan type compatibility, serta mencetak diagnostic semantik. Validasi yang telah tersedia mencakup pengecekan deklarasi, scope, tipe, assignment, control flow, array indexing, record field access, procedure/function call, built-in readln dan writeln, serta penggunaan variable sebelum inisialisasi. Output semantic report disimpan ke file test/milestone-3/<nama_input>_SEMANTIC.txt setelah parsing berhasil.

V. Saran

Pengembangan berikutnya dapat memperluas pengecekan String agar memperhatikan panjang literal sesuai spesifikasi, memperbaiki file input-3-type-error.txt agar langsung masuk tahap semantic analysis, dan menambahkan test untuk function return assignment serta argument procedure/function buatan pengguna. Output Decorated AST juga dapat dibuat lebih informatif dengan menampilkan status initialized langsung pada label node, bukan hanya pada symbol table.

VI. Lampiran

A. Link Repository Github

<https://github.com/alfarabi0642/PRX-Tubes-IF2224-2026>

B. Pembagian Tugas

NIM	Tugas
13524002	Parser dan lexer revisions untuk Milestone 3, termasuk validasi empty statement, comment filtering serta perubahan while, for agar wajib compound statement, dan dokumen bagian implementasi.
13524022	Implementasi AST model dan AST Builder, termasuk konversi parse tree menjadi AST ringkas untuk deklarasi, statement, expression, array, record, procedure, function call, dan testing milestone 3.
13524086	Implementasi type system dan symbol table (TypeRegistry, tab, btab, atab, predefined identifiers), serta integrasi dan refactoring modul semantic analysis.
13524094	Implementasi semantic analyzer, decorated AST, printer, diagnostics, integrasi main.cpp, refactoring akhir.

VII. Referensi

1. Tutorialspoint. "Compiler Design - Lexical Analysis."
https://www.tutorialspoint.com/compiler_design/compiler_design_lexical_analysis.html
2. J. E. Hopcroft, R. Motwani, dan J. D. Ullman. Introduction to Automata Theory, Languages, and Computation, 3rd ed.
https://cdn-edunex.itb.ac.id/29161-Formal-Language-Theory-and-Automata/1629640939613_Introduction-to-Automata-Theory,-Languages,-and-Computation-Edition-3.pdf
3. A. V. Aho, M. S. Lam, R. Sethi, dan J. D. Ullman. Compilers: Principles, Techniques, and Tools, 2nd ed. (Dragon Book), 2006.
[https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20\(2006\).pdf](https://repository.unikom.ac.id/48769/1/Compilers%20-%20Principles,%20Techniques,%20and%20Tools%20(2006).pdf)